

```
<app-demo></app-demo>
```

```
@Component({  
  selector: 'app-demo',  
  template: '<p>This is a paragraph</p>'  
})  
export class DemoComponent {  
  constructor() {  
    //Some code  
  }  
  
  title: string = 'App Demo';  
  show: boolean = false;  
  @input() myVar: string;  
}
```



When a constructor is called, by that time, none of its input properties are updated and available to use.

When a constructor is called, by that time, the child components of that component are not yet constructed.

Projected contents are also not available by the time the constructor of a component is called.

Once the component is removed from the DOM, we can say component is destroyed.

```
<app-demo>  
  <p>This content will be projected.</p>  
</app-demo>
```

```
@Component({  
  selector: 'app-demo',  
  template: '<h2>Content Projection</h2>  
            <ng-content></ng-content>  
            <p>This is a paragraph</p>',  
})  
export class DemoComponent{  
  constructor(){  
    //Some code  
  }  
  
  title: string = 'App Demo';  
  show: boolean = false;  
  @input() myVar: string;  
}
```



Component Created

Component Created

- When the Angular application starts, it first creates and renders the root component.
- Then it creates and renders its children and their children. In this way, it forms a tree of components.
- Once Angular loads the component, it starts the process of rendering the view. To do that, it needs to check the input properties, evaluate the data bindings & expressions, render the projected content etc.
- Angular then also removes the component from the DOM when it is no longer needed.
- And Angular lets us know when these events happen, using Angular lifecycle hooks.



Component Created

ngOnChanges

ngOnInit

ngDoCheck

ngAfterContentInit

ngAfterContentChecked

ngAfterViewInit

ngAfterViewChecked

ngDestroy

Component Created

The Angular **life cycle hooks** are the methods that angular invokes on a directive or a component, as it creates, changes and destroys them.



The Angular **life cycle hooks** are the methods that angular invokes on a directive or a component, as it creates, changes and destroys them.



Change detection in Angular is a mechanism by which, Angular keeps the view template in sync with the component class.

```
<div>Hello {{ name }}</div>
```

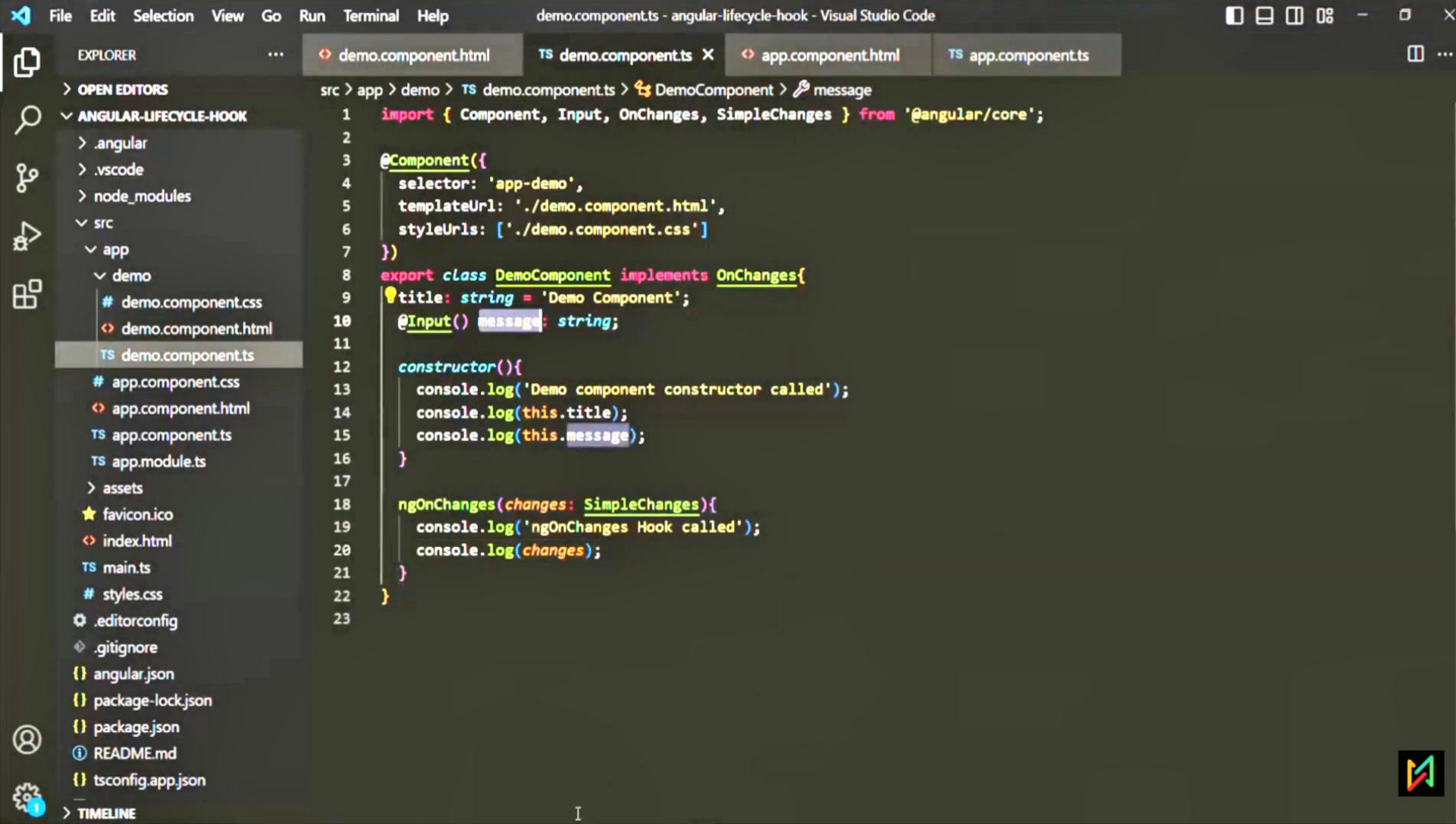
Whenever the @input property of a component changes

Whenever a DOM event happens. E.g. Click or Change

Whenever a timer events happens using `setTimeout()` / `setInterval()`.

Whenever an HTTP request is made.





EXPLORER

demo.component.html

TS demo.component.ts X

app.component.html

TS app.component.ts

> OPEN EDITORS

v ANGULAR-LIFECYCLE-HOOK

> .angular

> .vscode

> node_modules

v src

v app

v demo

demo.component.css

< demo.component.html

TS demo.component.ts

app.component.css

< app.component.html

TS app.component.ts

TS app.module.ts

> assets

★ favicon.ico

< index.html

TS main.ts

styles.css

⚙ .editorconfig

💎 .gitignore

{ } angular.json

{ } package-lock.json

{ } package.json

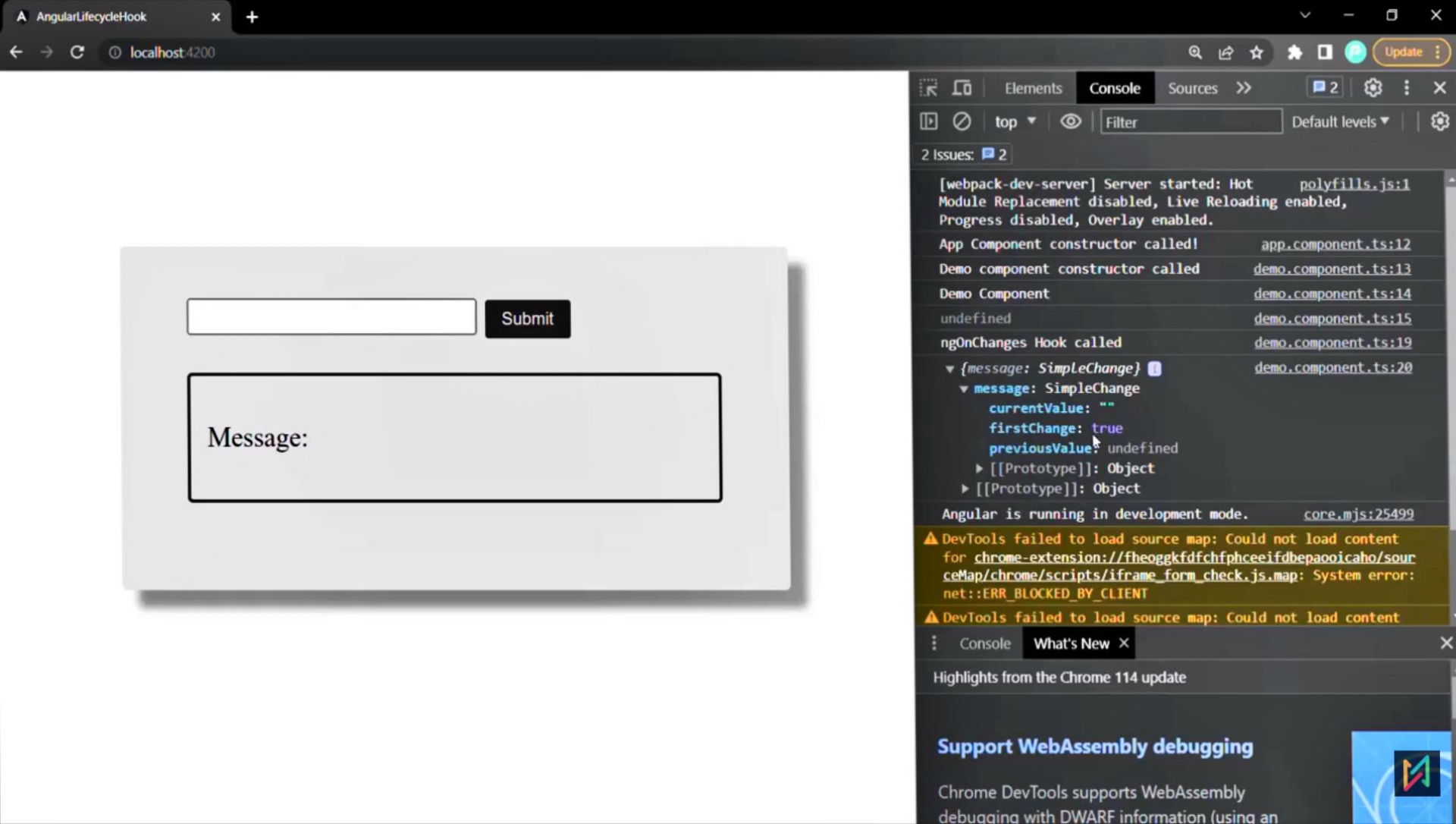
📖 README.md

{ } tsconfig.app.json

> TIMELINE

src > app > demo > TS demo.component.ts > DemoComponent > message

```
1  import { Component, Input, OnChanges, SimpleChanges } from '@angular/core';
2
3  @Component({
4    selector: 'app-demo',
5    templateUrl: './demo.component.html',
6    styleUrls: ['./demo.component.css']
7  })
8  export class DemoComponent implements OnChanges{
9    title: string = 'Demo Component';
10    @Input() message: string;
11
12    constructor(){
13      console.log('Demo component constructor called');
14      console.log(this.title);
15      console.log(this.message);
16    }
17
18    ngOnChanges(changes: SimpleChanges){
19      console.log('ngOnChanges Hook called');
20      console.log(changes);
21    }
22  }
23
```

Component Created

ngOnChanges

Component Created

The ngOnChanges hook get executed at the start, when a new component is created and its input bound properties are updated.

The ngOnChanges hook also gets executes everytime the input bound properties of the component changes

The ngOnChanges hook is not raised if the change detection cycle does not find any changes in the input propertie's value.





Angular raises `ngOnInit` hook after it creates the component and update its input properties. This hook is raised after `ngOnChanges`.

The `ngOnInit` hook is fired only once i.e. during the first change detection cycle. After that, if the input property changes, this hook does not get called.

By the time `ngOnInit` gets called, none of the child components or projected contents or view are available at this point. Hence any property decorated with `@ViewChild`, `@ViewChildren`, `@ContentChild` or `@ContentChildren` will not be available to use.





EXPLORER

> OPEN EDITORS

✓ ANGULAR-LIFECYCLE-HOOK

> .angular

> .vscode

> node_modules

✓ src

✓ app

✓ demo

demo.component.css

demo.component.html

TS demo.component.ts

app.component.css

app.component.html

TS app.component.ts

TS app.module.ts

> assets

★ favicon.ico

index.html

TS main.ts

styles.css

.editorconfig

.gitignore

{ } angular.json

{ } package-lock.json

{ } package.json

i README.md

{ } tsconfig.app.json

> TIMELINE

src > app > demo > TS demo.component.ts > DemoComponent > ngOnInit

```
1  import { Component, Input, OnChanges, SimpleChanges, OnInit, ViewChild, ElementRef } from '@angular/core';
2
3  @Component({
4    selector: 'app-demo',
5    templateUrl: './demo.component.html',
6    styleUrls: ['./demo.component.css']
7  })
8  export class DemoComponent implements OnChanges, OnInit{
9    title: string = 'Demo Component';
10    @Input() message: string;
11    @ViewChild('temp') tempPara: ElementRef;
12
13    constructor(){
14      console.log('Demo component constructor called');
15      // console.log(this.title);
16      // console.log(this.message);
17    }
18
19    ngOnChanges(changes: SimpleChanges){
20      console.log('ngOnChanges Hook called');
21      // console.log(changes);
22    }
23
24    ngOnInit(){
25      console.log('ngOnInit Hook called');
26      console.log(this.tempPara.nativeElement.innerHTML);
27    }
28  }
29
```





Angular invokes **ngDoCheck** hook during every change detection cycle. This hook is invoked even if there is no change in input bound properties.

For example: The **ngDoCheck** lifecycle hook will run if you clicked a button on webpage, which does not do anything. But still its an event so the change detection cycle will run and execute **ngDoCheck** hook.

Angular invokes **ngDoCheck** lifecycle hook after **ngOnChanges** & **ngOnInit** hooks.

You can use this hook to implement a custom change detection, whenever angular fails to detect any change made to input bound properties.

The **ngDoCheck** hook is also a great place to use, when you want to execute some code on every change detection cycle.





The `ngAfterContentInit` lifecycle hook is called after the components projected content has been fully initialized.

parent.component.html

```
<h2>Parent Component</h2>
<app-child>
  <h2>Some Heading</h2>
  <p #paragraph>This is a paragraph</p>
  <app-test></app-test>
</app-child>
```

child.component.html

```
<h2>Parent Component</h2>
<ng-content></ng-content>
```



Component Created

ngOnChanges

ngOnInit

ngDoCheck

ngAfterContentInit



Component Created

The `ngAfterContentInit` lifecycle hook is called after the components projected content has been fully initialized.

Angular updates the properties decorated with `@ContentChild` & `@ContentChildren` decorator just before this hook is raised.

This lifecycle hook gets called only once, during the first change detection cycle. After that, if the projected content changes, this lifecycle hook will not get called.





The **ngAfterContentChecked** lifecycle hook is called during every change detection cycle, after Angular has finished initializing and checking projected content.

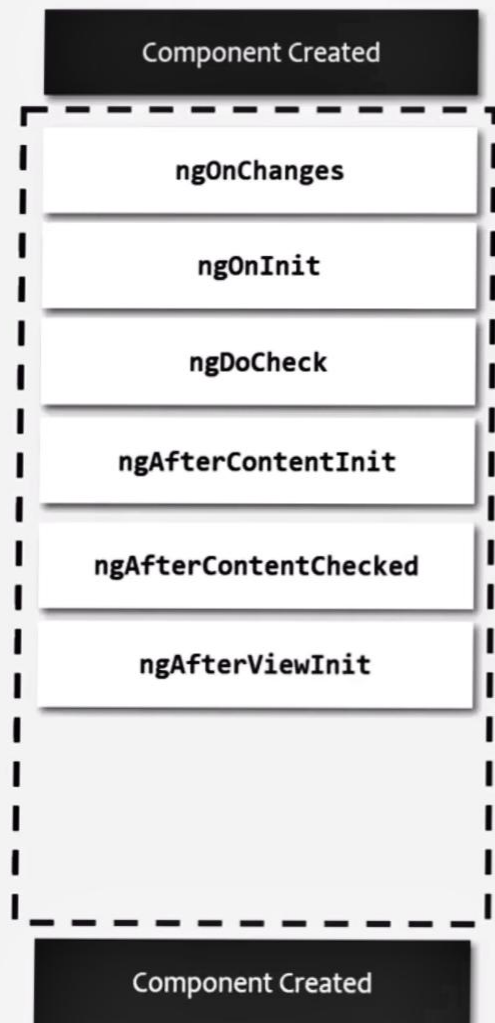
Angular also updates the properties decorated with **@ContentChild** & **@ContentChildren** decorator, before raising **ngAfterContentChecked** hook.

Angular raises this hook even if there is no projected content in the component.

The **ngAfterContentInit** hook is called after the projected content is initialized. **ngAfterContentChecked** is called whenever the projected content is *initialized, checked & updated*.

NOTE: The **ngAfterContentInit** & **ngAfterContentChecked** are component only hook. These hooks are not available for directives.





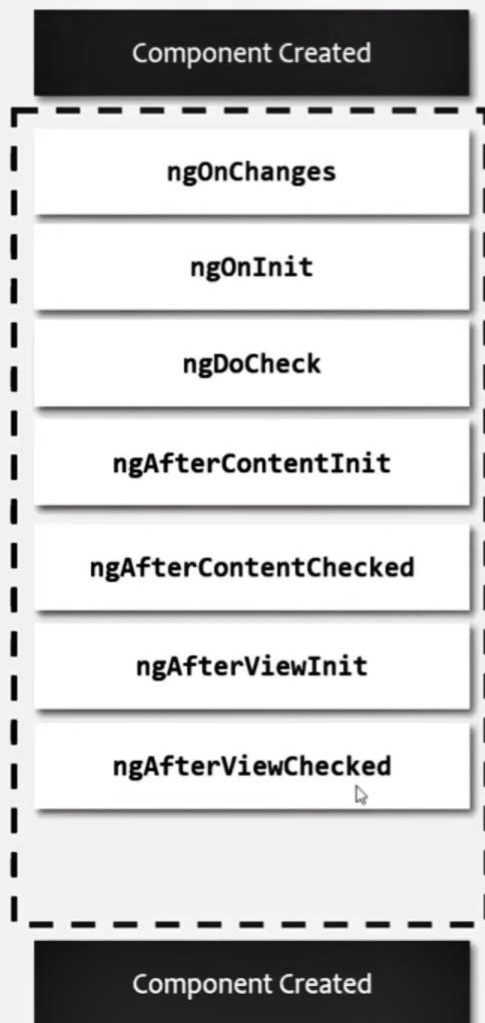
The **`ngAfterViewInit`** is called after the components View template and all its child components view templates are fully initialized.

Angular also updates the properties decorated with **`@ViewChild`** and **`@ViewChildren`** decorator before raising this hook.

This hook is called during the first change detection cycle, when Angular initializes the view for the first time.

By the time this hook gets called for a component, all the lifecycle hook methods of child components and directives are completely processed and child components are completely ready.





Angular fires **ngAfterViewChecked** hook after it checks and updates the components View template and all its child components view templates.

This hook is called during the first change detection cycle, after **ngAfterViewInit** hook has executed. And after that during every change detection cycle.

Angular also updates the properties decorated with **@ViewChild** and **@ViewChildren** decorator before raising this hook.

NOTE: The **ngAfterViewChecked** hook is a component only hook. It is not available for directives.



Component Created

`ngOnChanges`

`ngOnInit`

`ngDoCheck`

`ngAfterContentInit`

`ngAfterContentChecked`

`ngAfterViewInit`

`ngAfterViewChecked`

`ngOnDestroy`

Component Destroyed

Angular fires **`ngOnDestroy`** lifecycle hook just before the component or the directive is destroyed i.e. removed from the DOM.

This hook is a great place to do some cleanup work like unsubscribe from an observable or detach event handler etc., as this hook is called right before the component is destroyed.

This **`ngOnDestroy`** is the last lifecycle hook of a component & a directive.



What is a pipe?

Pipes in Angular are used to transform or format data before displaying it in the view. Angular pipe takes data as an input and it formats or transforms that data before displaying it in the template.



1

File

Edit

Selection

View

Go

...

←

→

angular-pipes

0%

-

×

EXPLORER

...

OPEN EDITORS

1 unsaved

admin.component.html...

ANGULAR-PIPES

> .angular

> .vscode

> node_modules

> src

> app

> admin

> admin.component.html

TS admin.component.ts

> Models

> Services

TS student.service.ts

> app.component.html

TS app.component.ts

TS app.module.ts

> assets

★ favicon.ico

> index.html

TS main.ts

styles.css

TIMELINE

admin.component.html

src > app > admin > admin.component.html > div.main-admin-container > div.admin-content > table > tr > td

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

<td *ngIf="!isEditing || std.id !== stdIdToEdit">{{ std.dob }}</td>

<td *ngIf="isEditing && std.id === stdIdToEdit">

<input type="date" [value]="std.dob" #editDob>

</td>

<td *ngIf="!isEditing || std.id !== stdIdToEdit">{{ std.course | uppercase }}</td>

<td *ngIf="isEditing && std.id === stdIdToEdit">

<select name="course" class="select-gender-course" [value]="std.course" #editCourse>

<option value="MBA">MBA</option>

<option value="B.Tech">B.TECH</option>

<option value="M.Sc">M.SC</option>

</select>

</td>

<td *ngIf="!isEditing || std.id !== stdIdToEdit">{{ std.marks }}</td>

<td *ngIf="isEditing && std.id === stdIdToEdit">

<input type="number" min="0" max="600" [value]="std.marks" #editMarks>

</td>

<td>{{ std.marks / totalMarks }}</td>

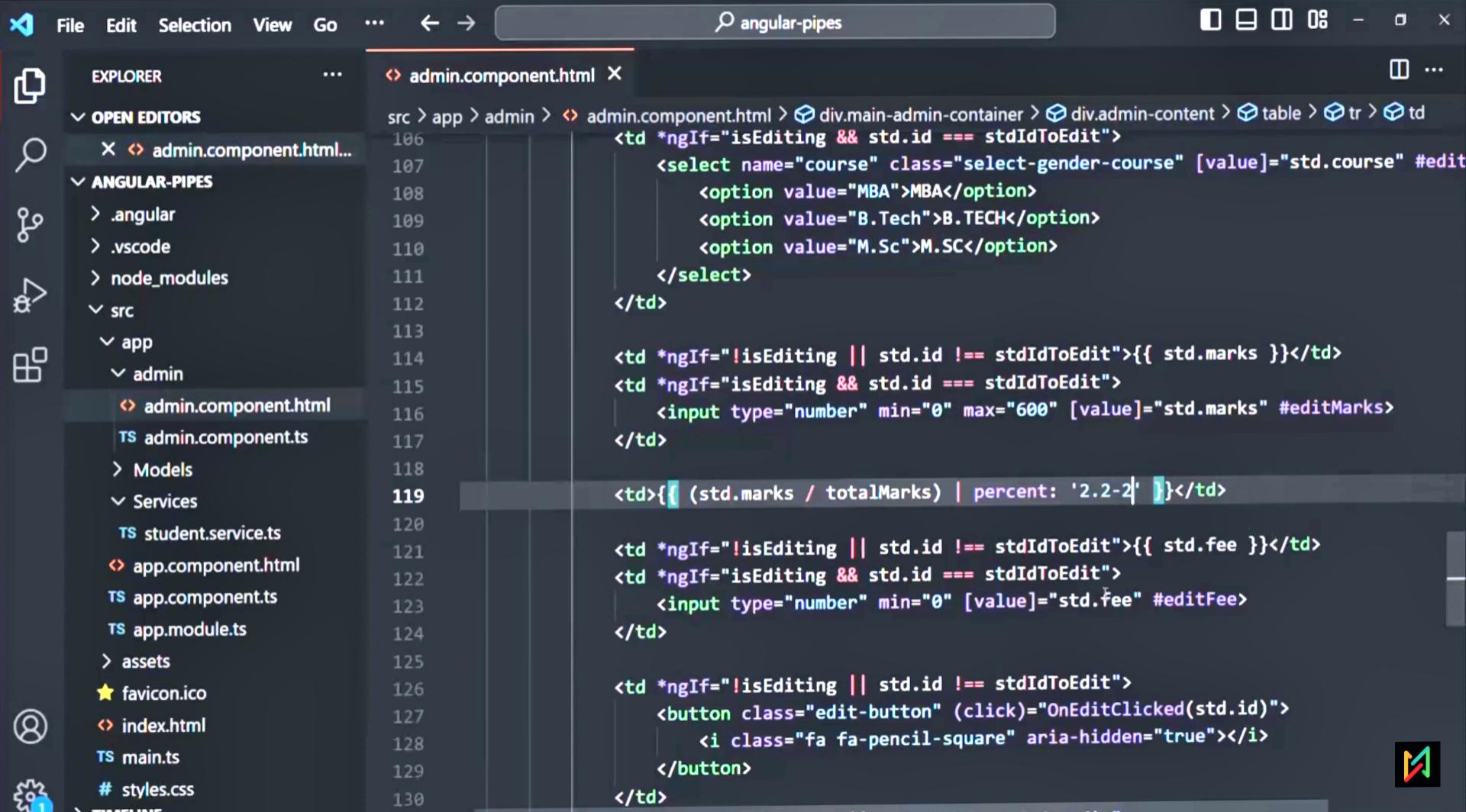
<td *ngIf="!isEditing || std.id !== stdIdToEdit">{{ std.fee }}</td>

<td *ngIf="isEditing && std.id === stdIdToEdit">

<input type="number" min="0" [value]="std.fee" #editFee>

</td>

pipe



Student Management

All



ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees		+
1	John Smith	Male	November 12, 1997	MBA	520	86.67%	1899	✓	✗
2	Mark Vought	Male	October 6, 1998	B.TECH	420	70.00%	2899	✓	✗
3	Sarah King	Female	September 22, 1996	B.TECH	540	90.00%	2899	✓	✗
4	Merry Jane	Female	June 12, 1995	MBA	380	63.33%	1899	✓	✗
5	Steve Smith	Male	December 21, 1999	M.SC	430	71.67%	799	✓	✗
6	Jonas Weber	Male	June 18, 1997	M.SC	320	53.33%	799	✓	✗



File

Edit

Selection

View

Go

...

←

→

angular-pipes

EXPLORER

...

OPEN EDITORS

admin.component.html...

ANGULAR-PIPES

.angular

.vscode

node_modules

src

app

admin

admin.component.html

TS admin.component.ts

Models

Services

TS student.service.ts

app.component.html

TS app.component.ts

TS app.module.ts

assets

★ favicon.ico

index.html

TS main.ts

styles.css

TIMELINE

admin.component.html

src > app > admin > admin.component.html > div.main-admin-container > div.admin-content > table > tr > td

106 <td *ngIf="isEditing && std.id === stdIdToEdit">

107 <select name="course" class="select-gender-course" [value]="std.course" #edit

108 <option value="MBA">MBA</option>

109 <option value="B.Tech">B.TECH</option>

110 <option value="M.Sc">M.SC</option>

111 </select>

112 </td>

113

114 <td *ngIf="!isEditing || std.id !== stdIdToEdit">{{ std.marks }}</td>

115 <td *ngIf="isEditing && std.id === stdIdToEdit">

116 <input type="number" min="0" max="600" [value]="std.marks" #editMarks>

117 </td>

118

119 <td>{{ (std.marks / totalMarks) | percent: '2.2-5' }}</td>

120

121 <td *ngIf="!isEditing || std.id !== stdIdToEdit">{{ std.fee }}</td>

122 <td *ngIf="isEditing && std.id === stdIdToEdit">

123 <input type="number" min="0" [value]="std.fee" #editFee>

124 </td>

125

126 <td *ngIf="!isEditing || std.id !== stdIdToEdit">

127 <button class="edit-button" (click)="OnEditClicked(std.id)">

128 <i class="fa fa-pencil-square" aria-hidden="true"></i>

129 </button>

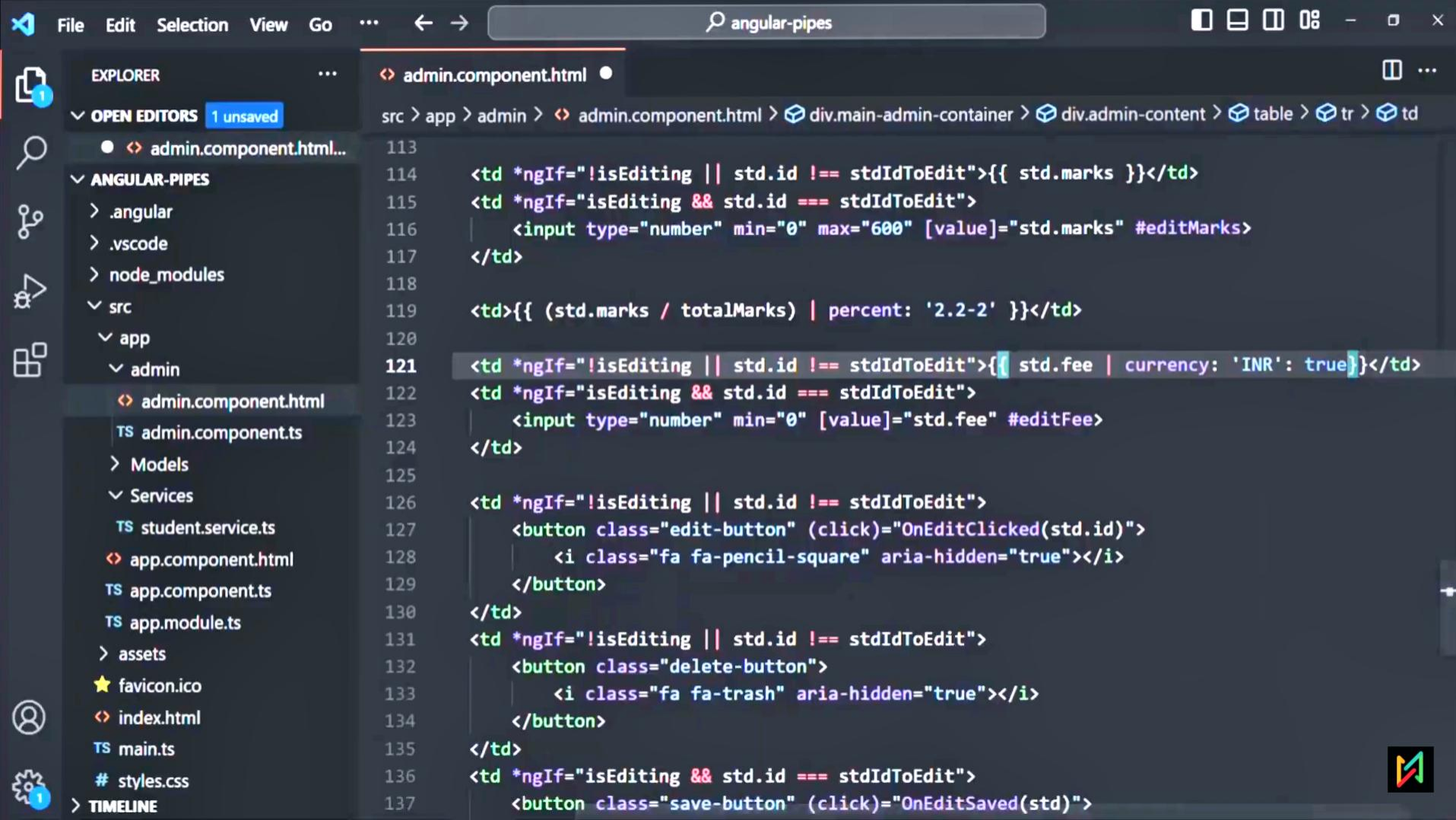
130 </td>

Student Management

All

ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees		+
1	John Smith	Male	November 12, 1997	MBA	520	86.66667%	1899	✓	✗
2	Mark Vought	Male	October 6, 1998	B.TECH	420	70.00%	2899	✓	✗
3	Sarah King	Female	September 22, 1996	B.TECH	540	90.00%	2899	✓	✗
4	Merry Jane	Female	June 12, 1995	MBA	380	63.33333%	1899	✓	✗
5	Steve Smith	Male	December 21, 1999	M.SC	430	71.66667%	799	✓	✗
6	Jonas Weber	Male	June 18, 1997	M.SC	320	53.33333%	799	✓	✗



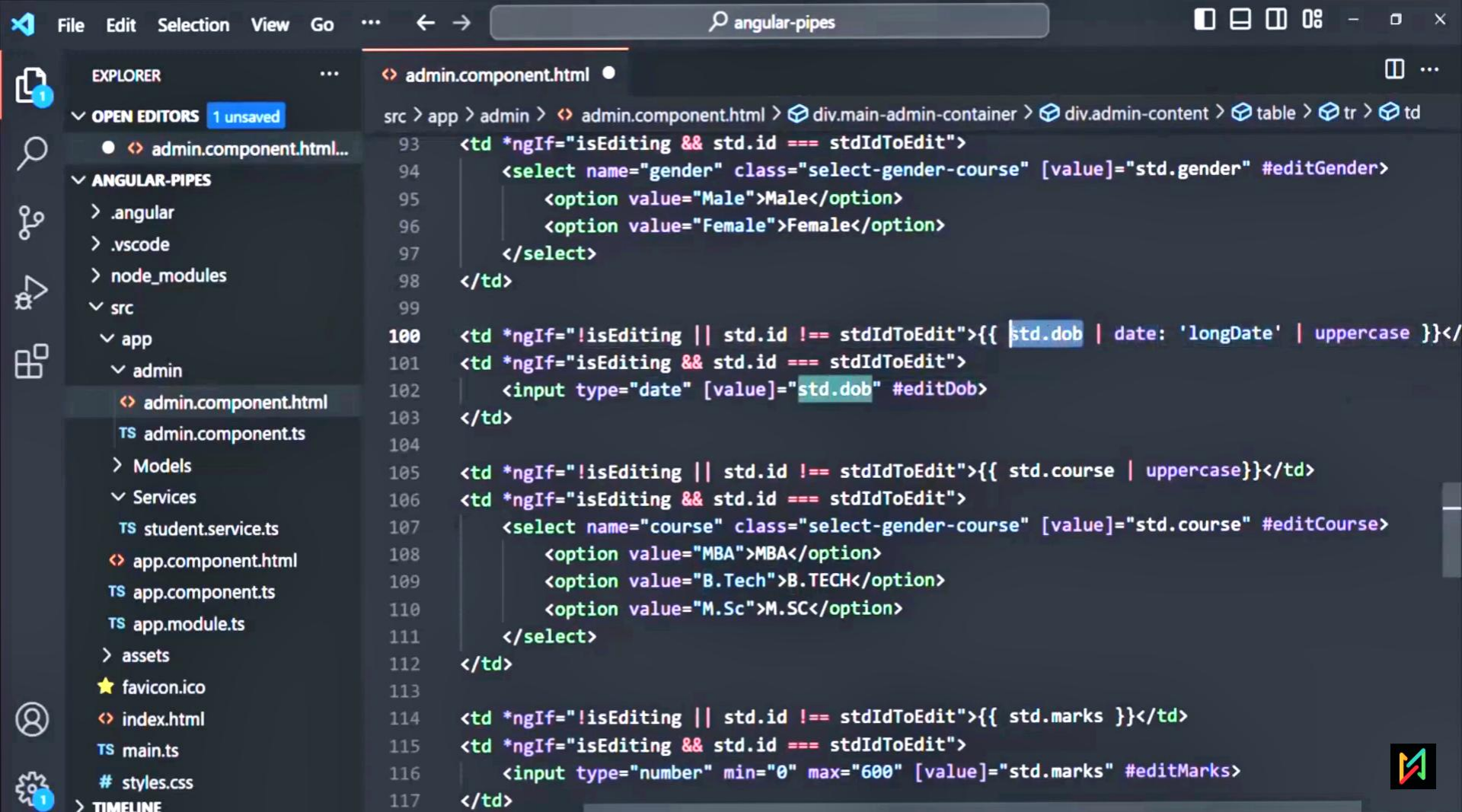


Student Management

All ▾

ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees		+
1	John Smith	Male	November 12, 1997	MBA	520	86.67%	₹1,899.00	✓	✗
2	Mark Vought	Male	October 6, 1998	B.TECH	420	70.00%	₹2,899.00	✓	✗
3	Sarah King	Female	September 22, 1996	B.TECH	540	90.00%	₹2,899.00	✓	✗
4	Merry Jane	Female	June 12, 1995	MBA	380	63.33%	₹1,899.00	✓	✗
5	Steve Smith	Male	December 21, 1999	M.SC	430	71.67%	₹799.00	✓	✗
6	Jonas Weber	Male	June 18, 1997	M.SC	320	53.33%	₹799.00	✓	✗



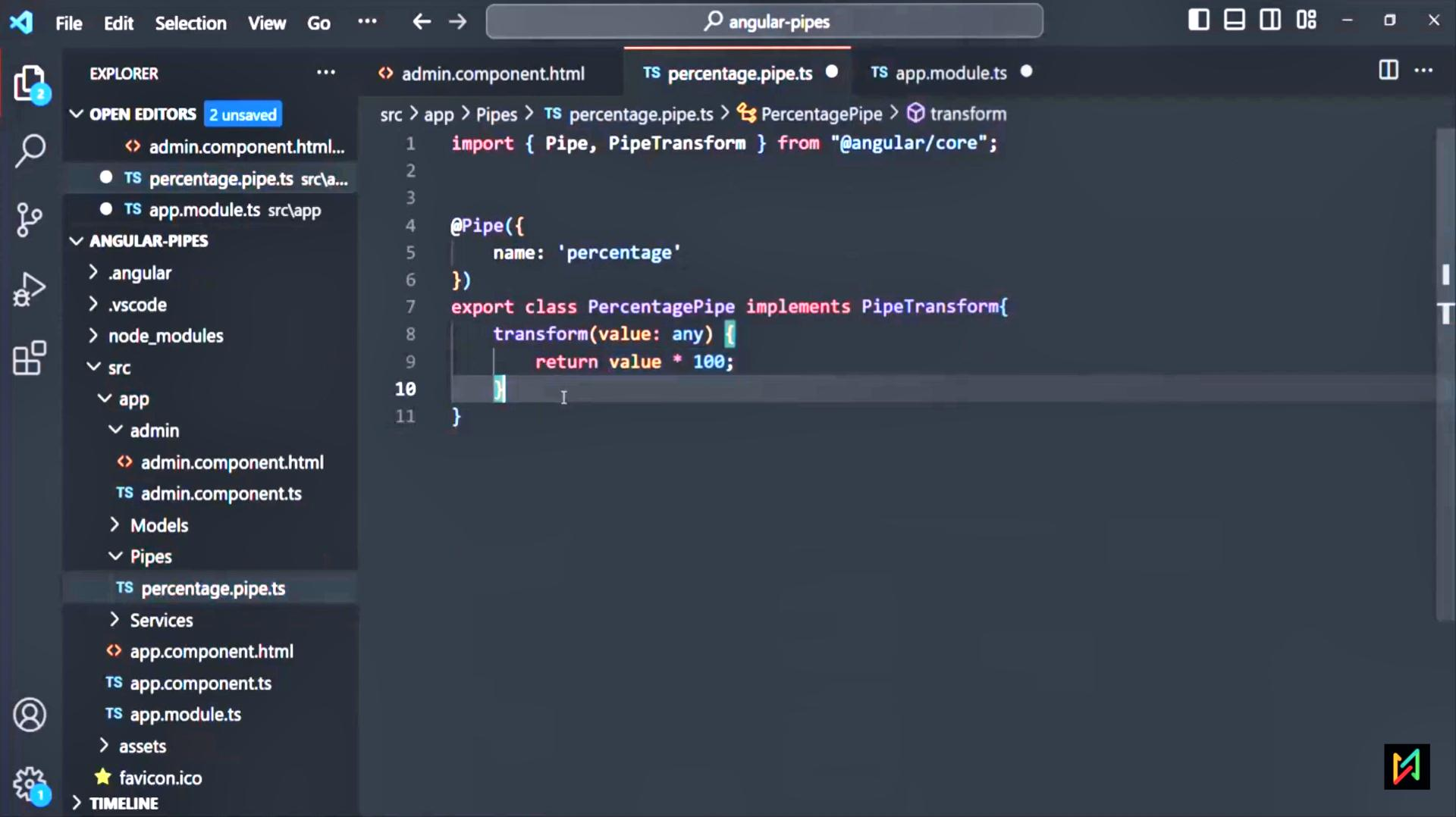


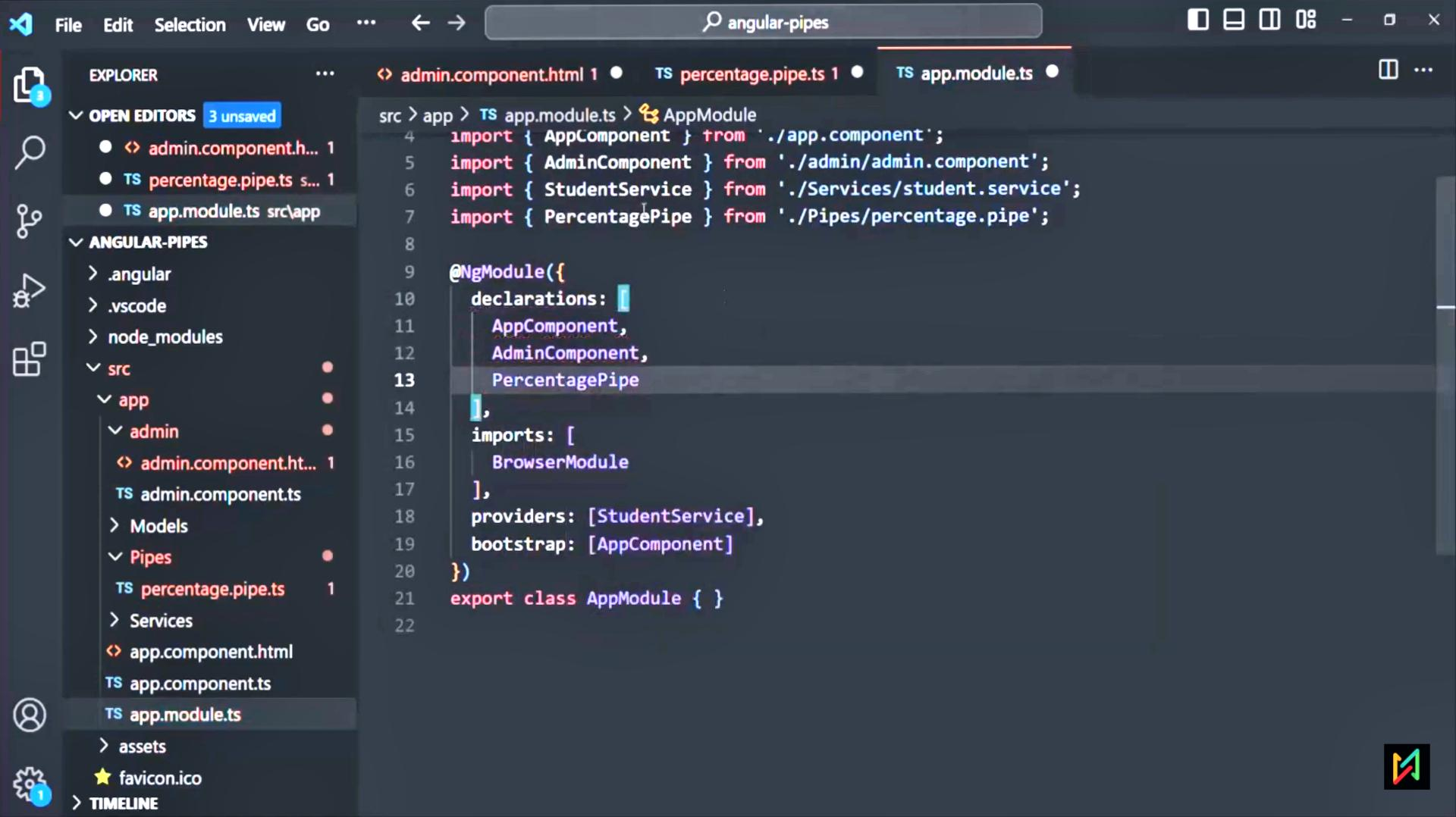
How to create a custom pipe?

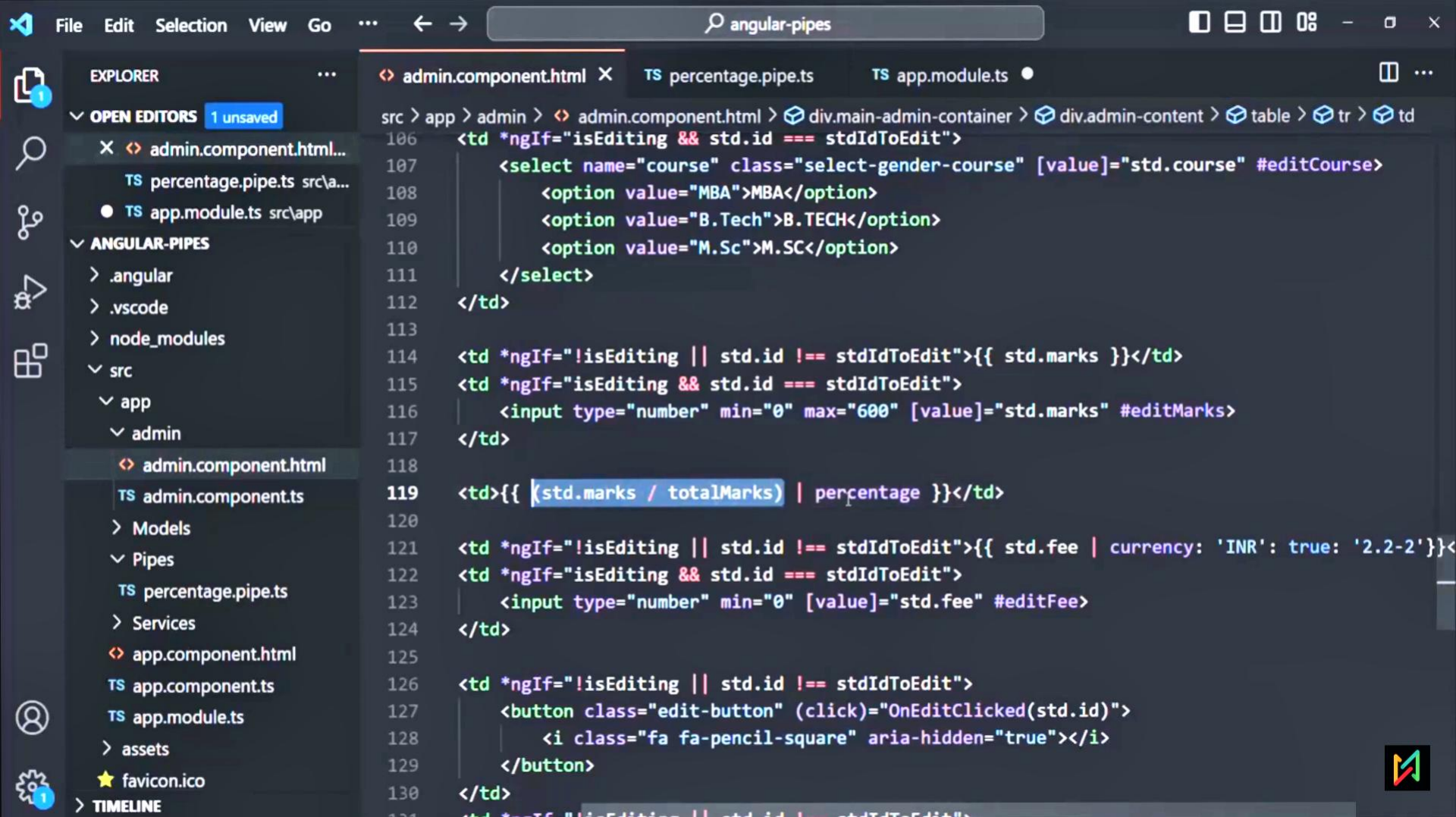
We can create a custom angular pipe in three simple steps:

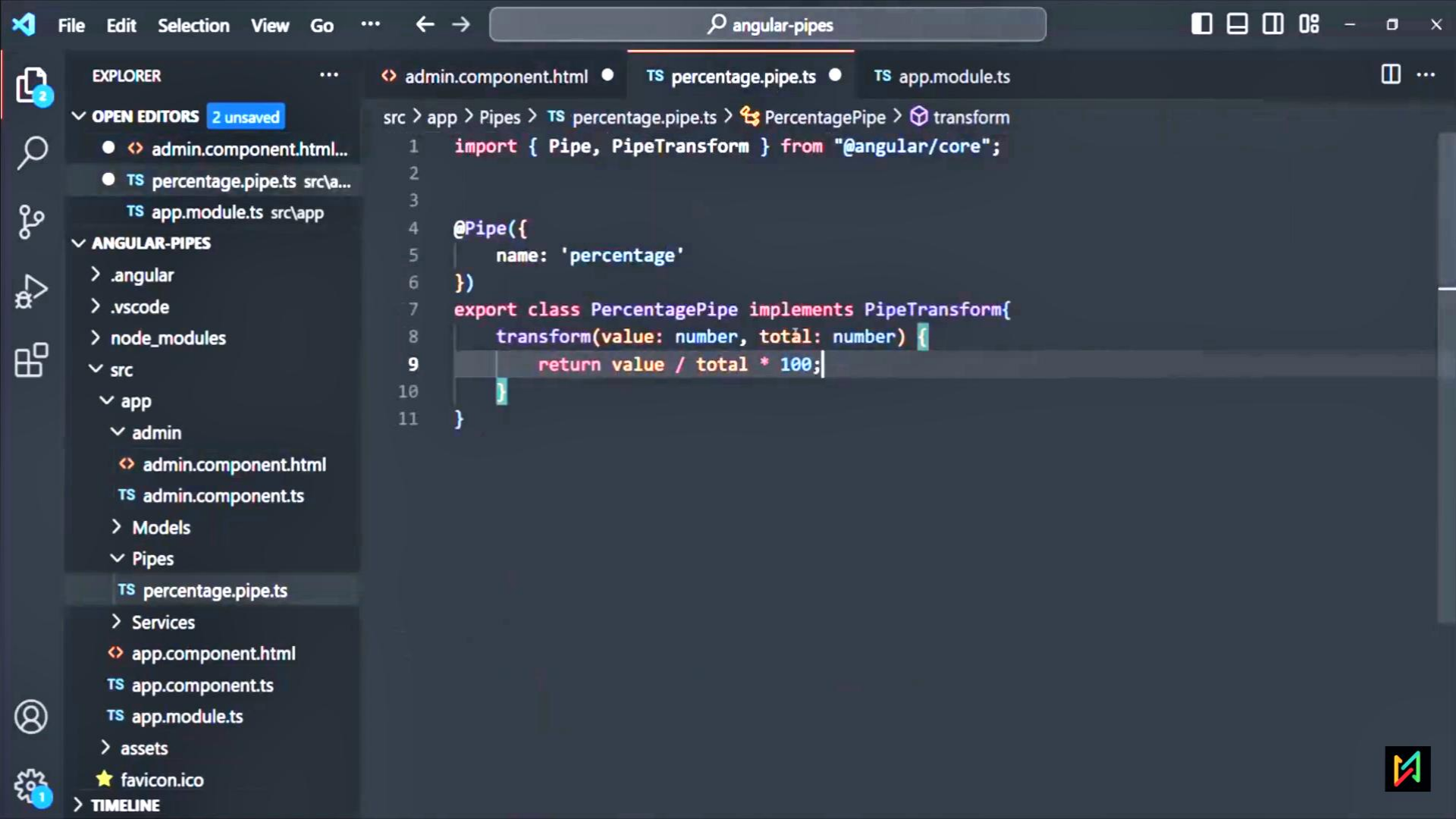
- 1 Create a typescript class and export it. By convention, a pipe class name should end with Pipe.
- 2 Decorate that class with `@Pipe` Decorator. There we can specify a name for the pipe.
- 3 Inherit `PipeTransform` interface and implement its `transform` method.

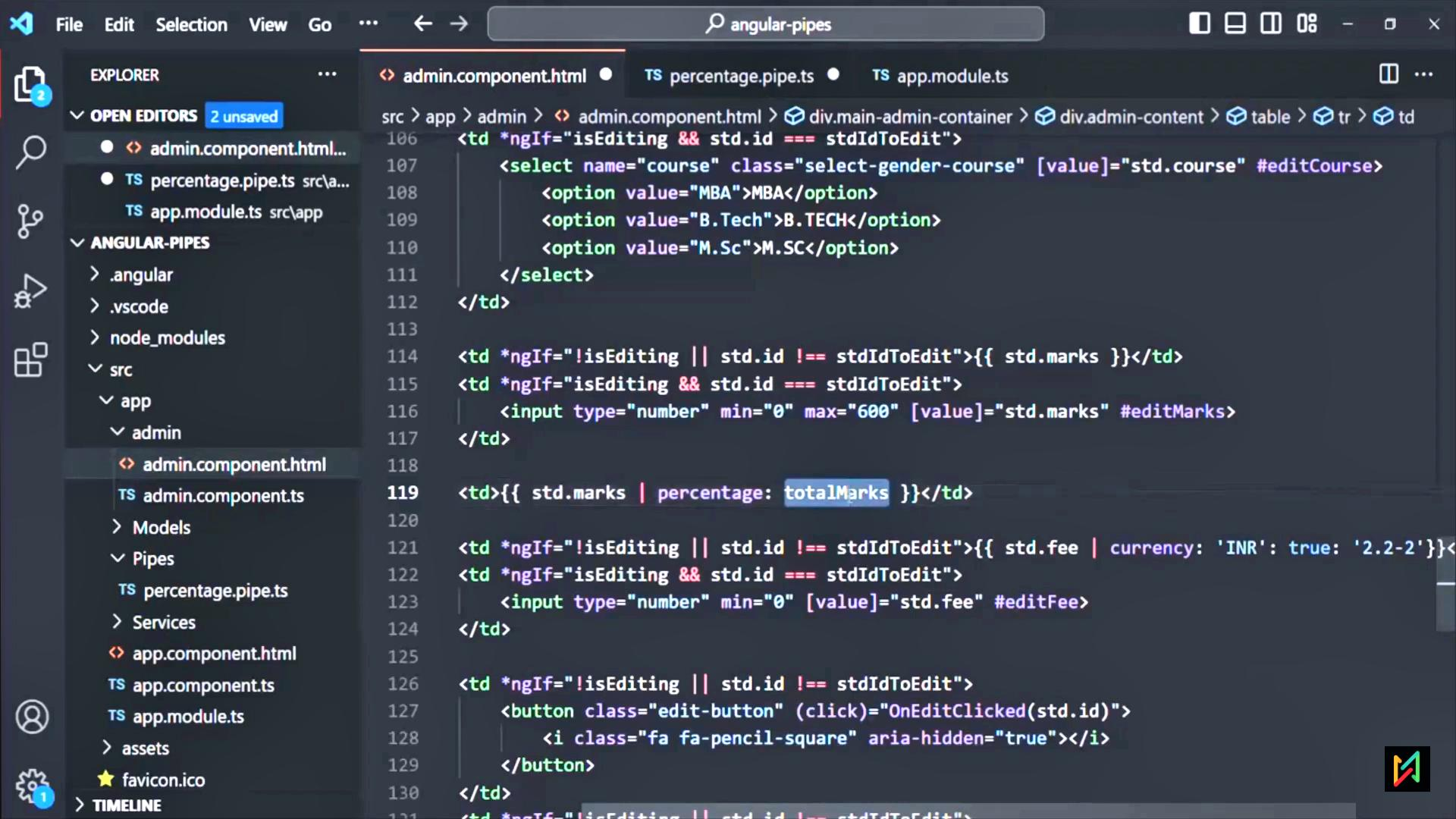


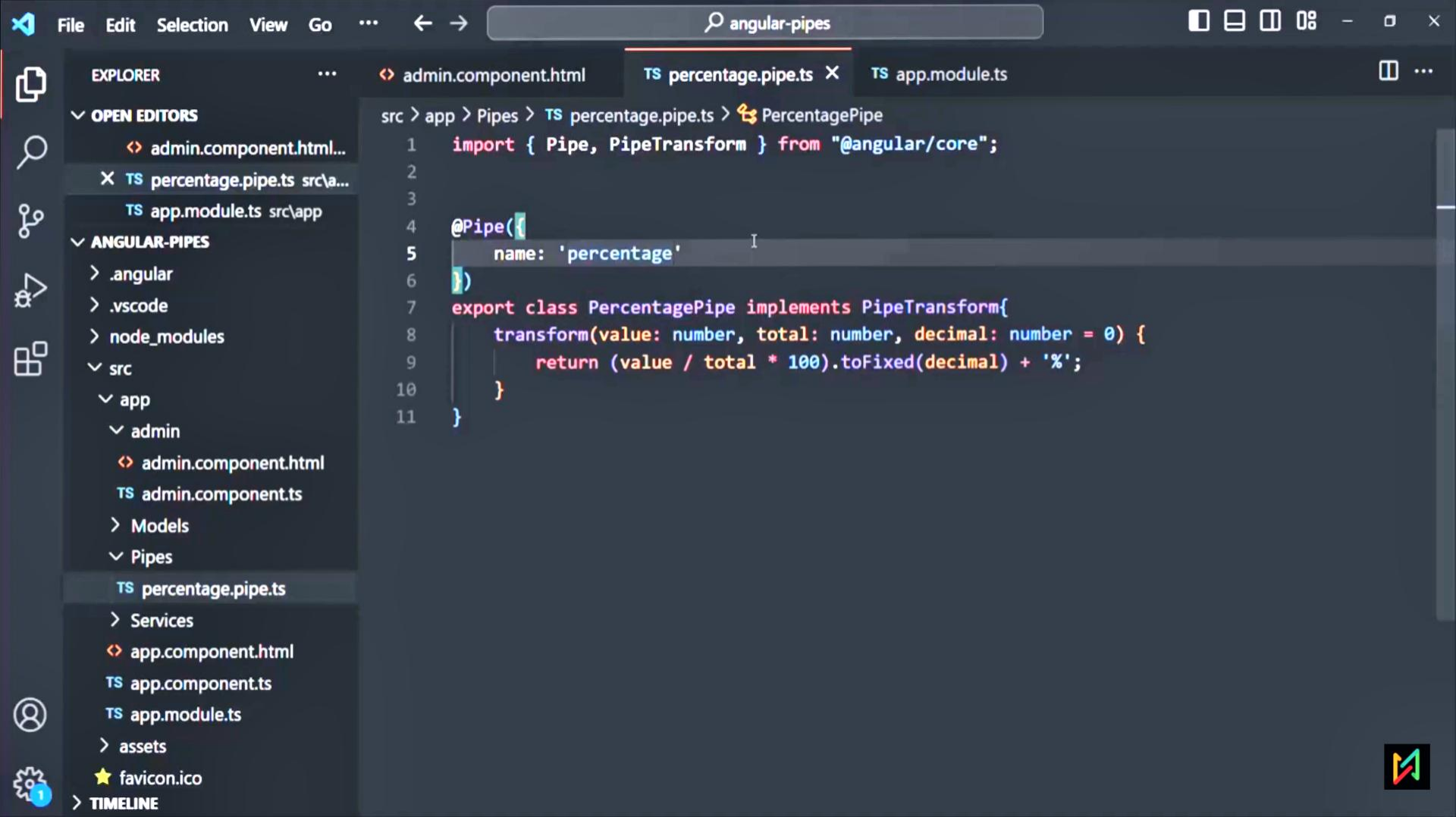


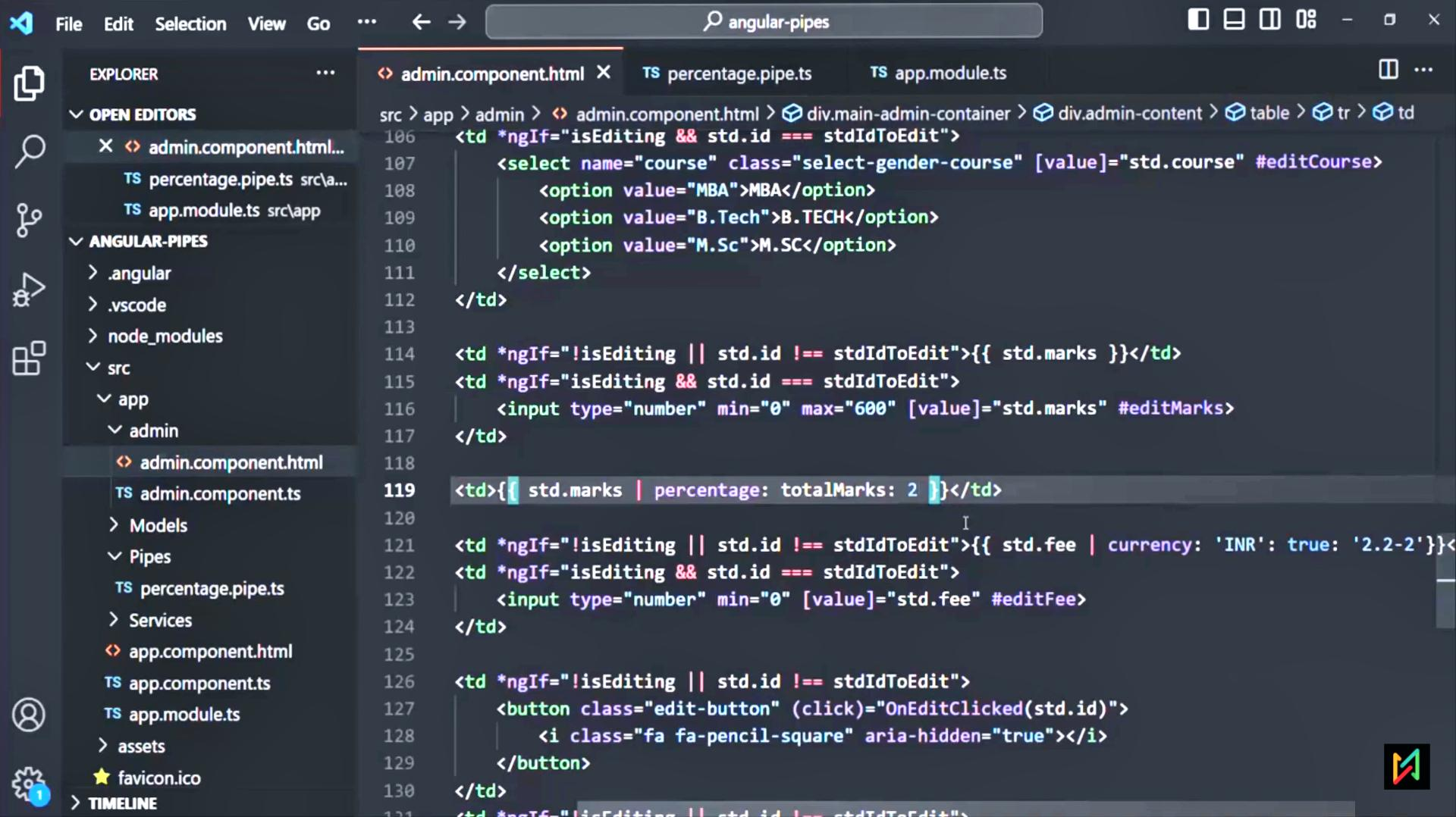












Student Management

All

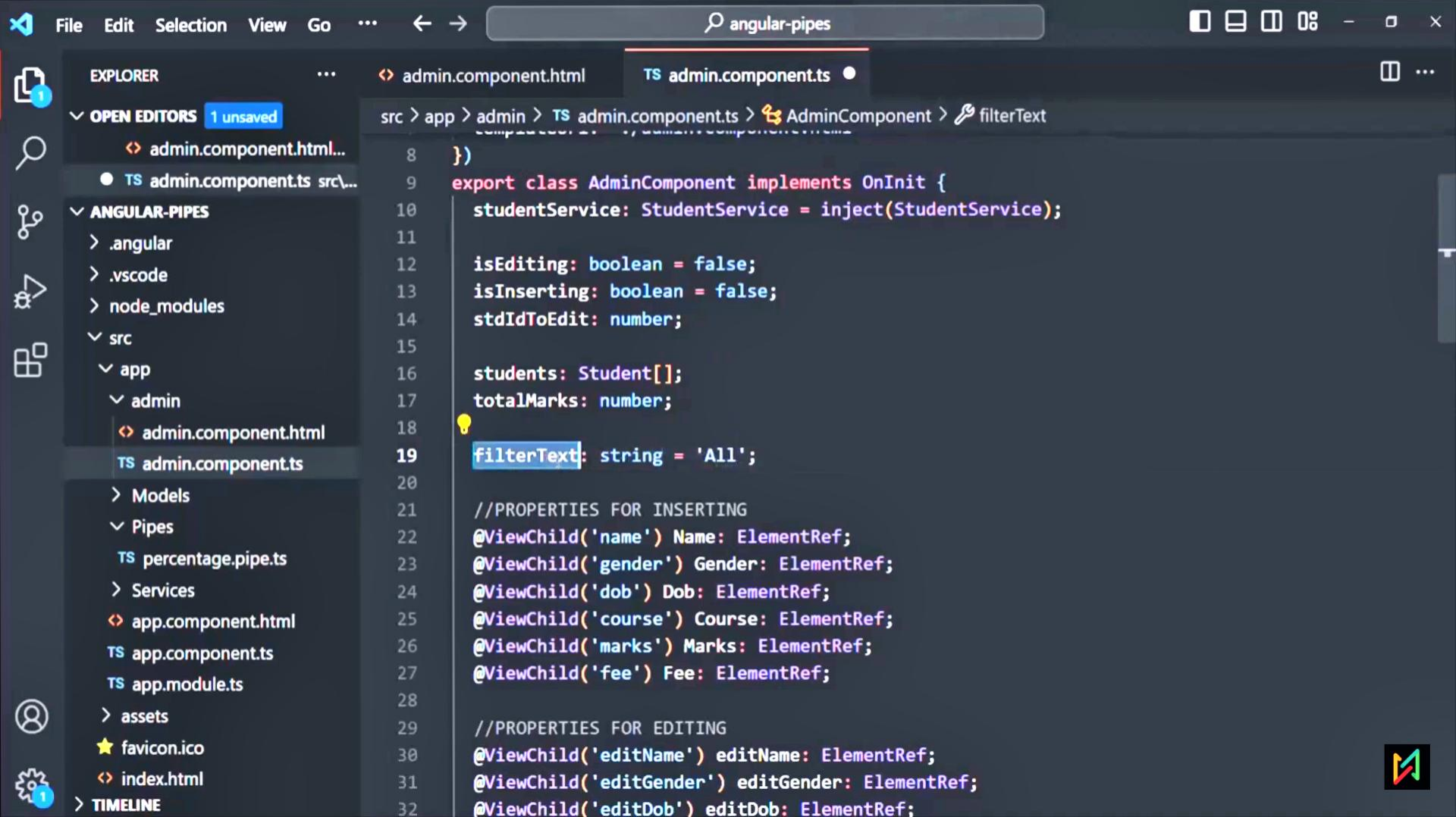
ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees		+
1	John Smith	Male	NOVEMBER 12, 1997	MBA	520	86.67%	₹1,899.00	✓	✗
2	Mark Vought	Male	OCTOBER 6, 1998	B.TECH	420	70.00%	₹2,899.00	✓	✗
3	Sarah King	Female	SEPTEMBER 22, 1996	B.TECH	540	90.00%	₹2,899.00	✓	✗
4	Merry Jane	Female	JUNE 12, 1995	MBA	380	63.33%	₹1,899.00	✓	✗
5	Steve Smith	Male	DECEMBER 21, 1999	M.SC	430	71.67%	₹799.00	✓	✗
6	Jonas Weber	Male	JUNE 18, 1997	M.SC	320	53.33%	₹799.00	✓	✗

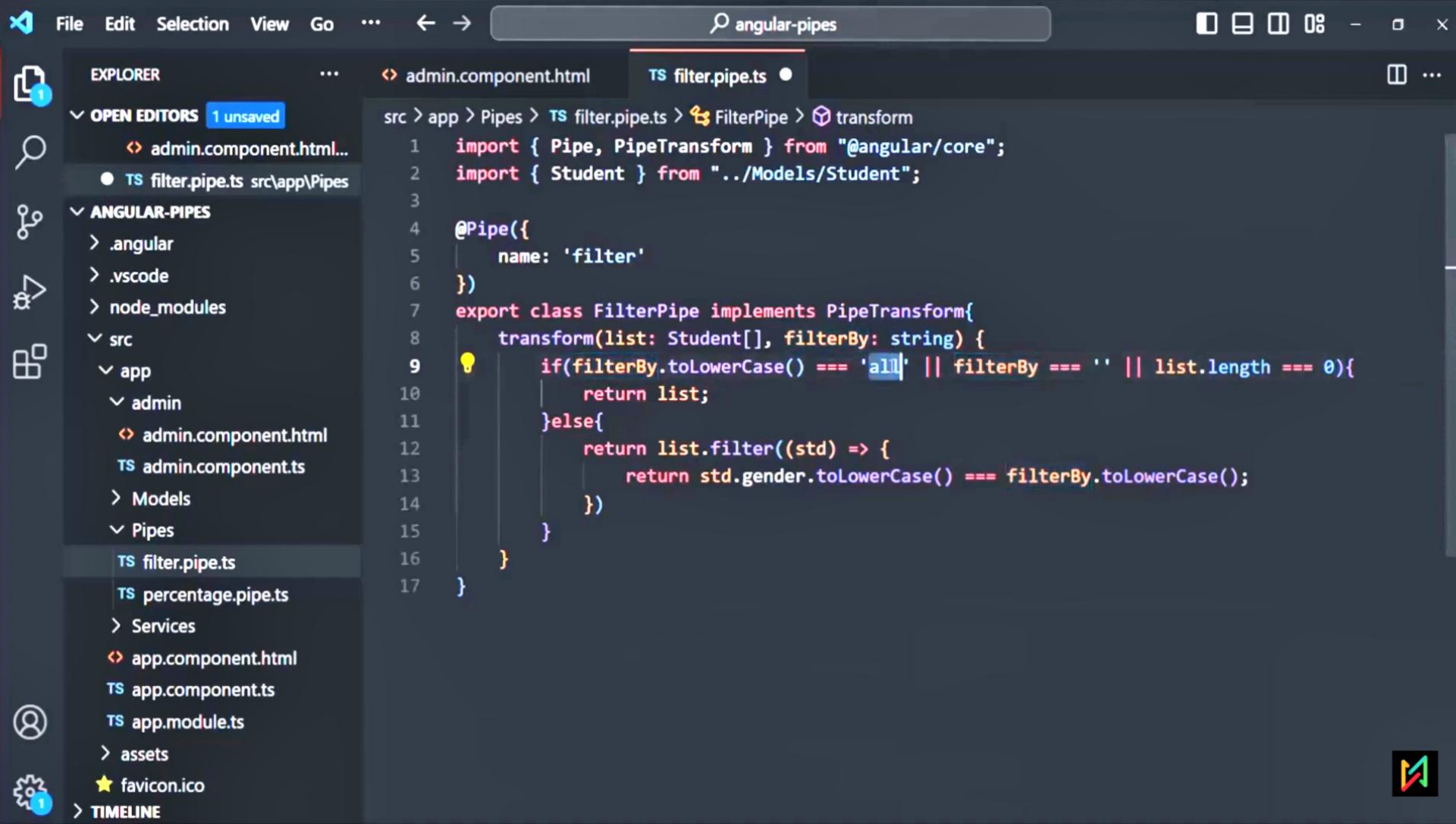
All rights reserved by procademy. @Copyright 2023.

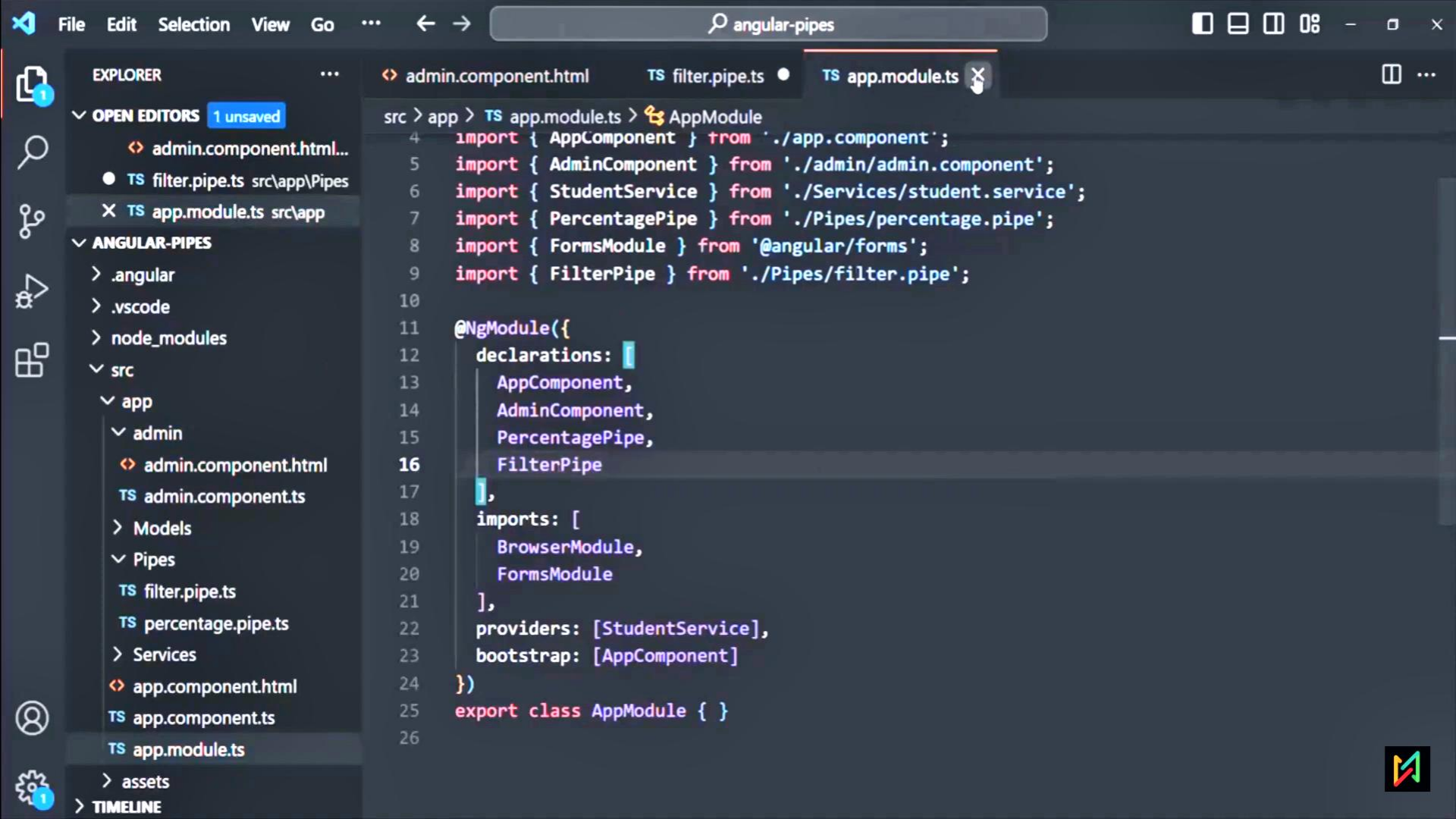


Creating a Custom Filter Pipes









File

Edit

Selection

View

Go

...

←

→

angular-pipes

0%

1

EXPLORER

...

OPEN EDITORS

1 unsaved

admin.component.html...

TS admin.component.ts src\...

TS filter.pipe.ts src\app\Pipes

ANGULAR-PIPES

> .angular

> .vscode

> node_modules

> src

> app

> admin

> admin.component.html

TS admin.component.ts

> Models

> Pipes

TS filter.pipe.ts

TS percentage.pipe.ts

> Services

> app.component.html

TS app.component.ts

TS app.module.ts

> assets

TIMELINE

admin.component.html

TS admin.component.ts

TS filter.pipe.ts

src > app > admin >

admin.component.html > div.main-admin-container > div.admin-content > table > tr

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

<!--DISPLAY ALL STUDENTS FROM STUDENTS ARRAY-->

<tr *ngFor="let std of students | filter: filterText">

<td>{{ std.id }}</td>

<td *ngIf="!isEditing || std.id !== stdIdToEdit">{{ std.name }}</td>

<td *ngIf="isEditing && std.id === stdIdToEdit">

<input type="text" [value]="std.name" #editName>

</td>

<td *ngIf="!isEditing || std.id !== stdIdToEdit">{{ std.gender }}</td>

<td *ngIf="isEditing && std.id === stdIdToEdit">

<select name="gender" class="select-gender-course" [value]="std.gender" #ed

<option value="Male">Male</option>

<option value="Female">Female</option>

</select>

</td>

<td *ngIf="!isEditing || std.id !== stdIdToEdit">{{ std.dob | date: 'longDate'}}

<td *ngIf="isEditing && std.id === stdIdToEdit">

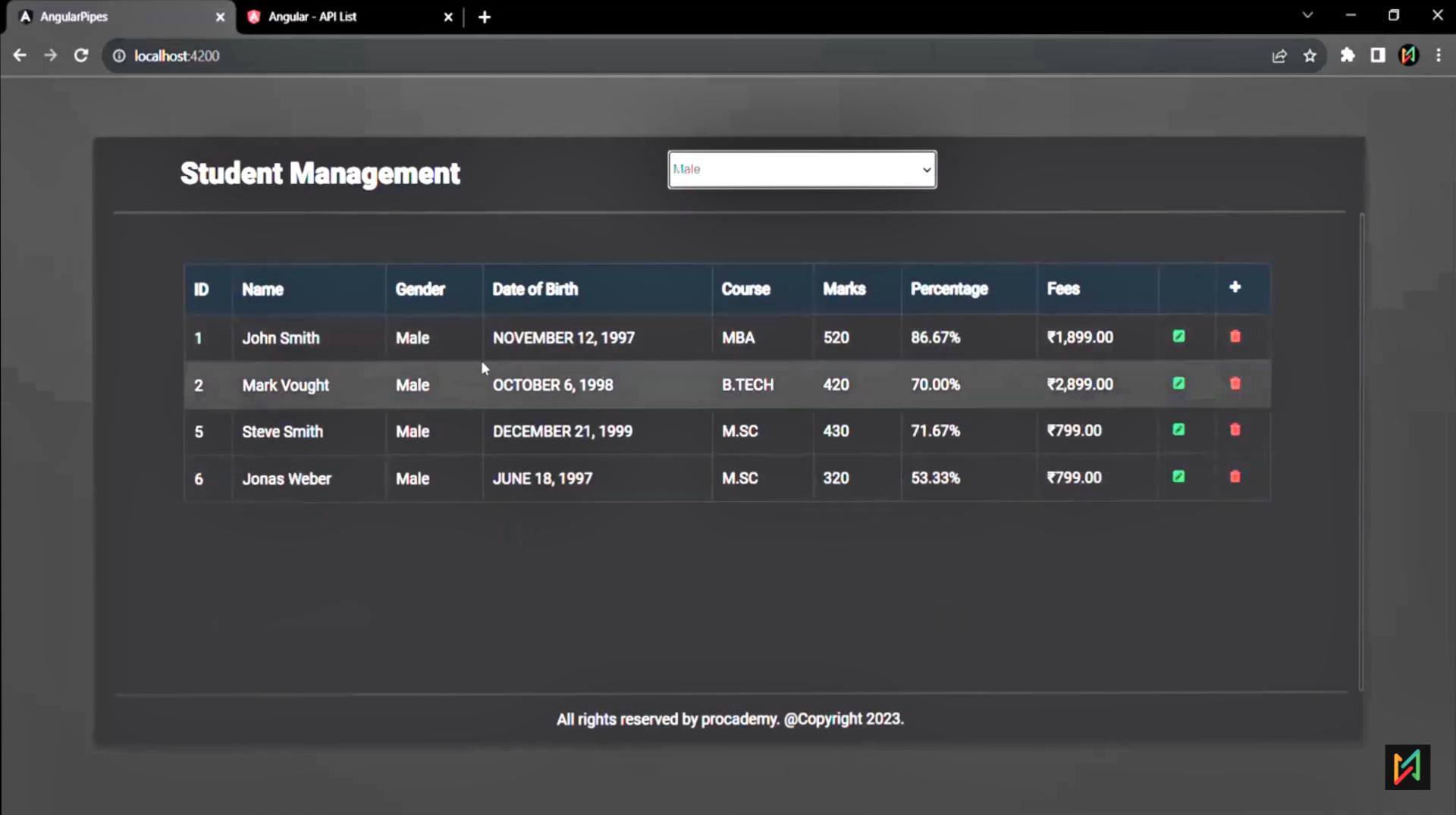
<input type="date" [value]="std.dob" #editDob>

</td>

<td *ngIf="!isEditing || std.id !== stdIdToEdit">{{ std.course | uppercase}}</td>

<td *ngIf="isEditing && std.id === stdIdToEdit">

<select name="course" class="select-gender-course" [value]="std.course" #ed



Student Management

Male

ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees		+
1	John Smith	Male	NOVEMBER 12, 1997	MBA	520	86.67%	₹1,899.00	✓	✕
2	Mark Vought	Male	OCTOBER 6, 1998	B.TECH	420	70.00%	₹2,899.00	✓	✕
5	Steve Smith	Male	DECEMBER 21, 1999	M.SC	430	71.67%	₹799.00	✓	✕
6	Jonas Weber	Male	JUNE 18, 1997	M.SC	320	53.33%	₹799.00	✓	✕

All rights reserved by procademy. @Copyright 2023.



Why to not use pipes for filtering or sorting?

Angular team always recommends not to use pipe to filter or sort data. When we use pipe for filtering or sorting data, it can significantly impact the performance of the application, if not implemented carefully.



Pure & Impure Pipes in Angular



What is a pure pipe?

A pure pipe is that pipe which gets called whenever there is a pure change on the input value. By default every pipe is a pure pipe.



What is pure change?

Following changes are considered as pure change in a data:

- 1 If the pipe is being used on a primitive type like string, number, boolean etc. And if the value of that primitive type changes, it is considered as a pure change
- 2 If the pipe is being used on a reference type input, and if the reference of that input changes, then the change is pure change.
- 3 **NOTE:** But if the input is of reference type, and only its property value changes and its reference has not changed, that change is not a pure change.



File

Edit

Selection

View

Go

...

←

→

angular-pipes

1

EXPLORER

...

admin.component.html

TS student.service.ts

TS percentage.pipe.ts

TS filter.pipe.ts

1 unsaved

admin.component.html...

TS student.service.ts src\ap...

TS percentage.pipe.ts src\ap...

TS filter.pipe.ts src\app\Pipes

ANGULAR-PIPES

> .angular

> .vscode

> node_modules

> src

> app

> admin

> admin.component.html

TS admin.component.ts

> Models

> Pipes

TS filter.pipe.ts

TS percentage.pipe.ts

> Services

TS student.service.ts

> app.component.html

TS app.component.ts

TIMELINE

src > app > Services > TS student.service.ts > StudentService > CreateStudent

```
1 import { Student } from "../Models/Student";
2
3 export class StudentService{
4     students: Student[] = [
5         new Student(1, 'John Smith', 'Male', new Date('11-12-1997'), 'MBA', 520, 1899),
6         new Student(2, 'Mark Vought', 'Male', new Date('10-06-1998'), 'B.Tech', 420, 2899),
7         new Student(3, 'Sarah King', 'Female', new Date('09-22-1996'), 'B.Tech', 540, 2899),
8         new Student(4, 'Merry Jane', 'Female', new Date('06-12-1995'), 'MBA', 380, 1899),
9         new Student(5, 'Steve Smith', 'Male', new Date('12-21-1999'), 'M.Sc', 430, 799),
10        new Student(6, 'Jonas Weber', 'Male', new Date('06-18-1997'), 'M.Sc', 320, 799),
11    ];//0x12345
12
13    totalMarks: number = 600;
14
15    CreateStudent(name, gender, dob, course, marks, fee){
16        let id = this.students.length + 1;
17        let student = new Student(id, name, gender, dob, course, marks, fee);
18        //this.students.push(student);
19
20        let studentCopy = [...this.students];//0xB4567
21        studentCopy.push(student);
22        this.students = studentCopy;
23    }
24 }
```


AngularPipes

Angular - API List

localhost:4200

Student Management

ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees		+
1	John Smith	Male	NOVEMBER 12, 1997	MBA	520	86.67%	₹1,899.00		
2	Mark Vought	Male	OCTOBER 6, 1998	B.TECH	420	70.00%	₹2,899.00		
3	Sarah King	Female	SEPTEMBER 22, 1996	B.TECH	540	90.00%	₹2,899.00		
4	Merry Jane	Female	JUNE 12, 1995	MBA	380	63.33%	₹1,899.00		
5	Steve Smith	Male	DECEMBER 21, 1999	M.SC	430	71.67%	₹799.00		
6	Jonas Weber	Male	JUNE 18, 1997	M.SC	320	53.33%	₹799.00		
7	TEST	Female	AUGUST 15, 2023	B.SC	320	53.33%	₹1,299.00		

Elements

Console

Sources

Network

Performance

Memory

Application

Security

Lighthouse

Recorder

Performance insights

top

Filter

Default levels

1 Issue

ap: System error: net::ERR_BLOCKED_BY_CLIENT

filter.pipe.ts:9

FILTER PIPE CALLED!

percentage.pipe.ts:9

PERCENTAGE PIPE CALLED!

admin.component.html:121

Warning: the currency pipe has been changed in Angular v5. The symbolDisplay option (third parameter) is now a string instead of a boolean. The accepted values are "code", "symbol" or "symbol-narrow".

What is an impure pipe?

A pipe is impure pipe if it is not pure. An impure pipe gets called for each change detection cycle and it is performance intensive.



1

File Edit Selection View Go ...

angular-pipes

EXPLORER

...

admin.component.html TS student.service.ts TS percentage.pipe.ts TS filter.pipe.ts

OPEN EDITORS 1 unsaved

admin.component.html... TS student.service.ts src\ap... TS percentage.pipe.ts src\app\ Pipes TS filter.pipe.ts

ANGULAR-PIPES

.angular .vscode node_modules src app admin admin.component.html admin.component.ts Models Pipes TS filter.pipe.ts TS percentage.pipe.ts Services TS student.service.ts app.component.html app.component.ts

TIMELINE

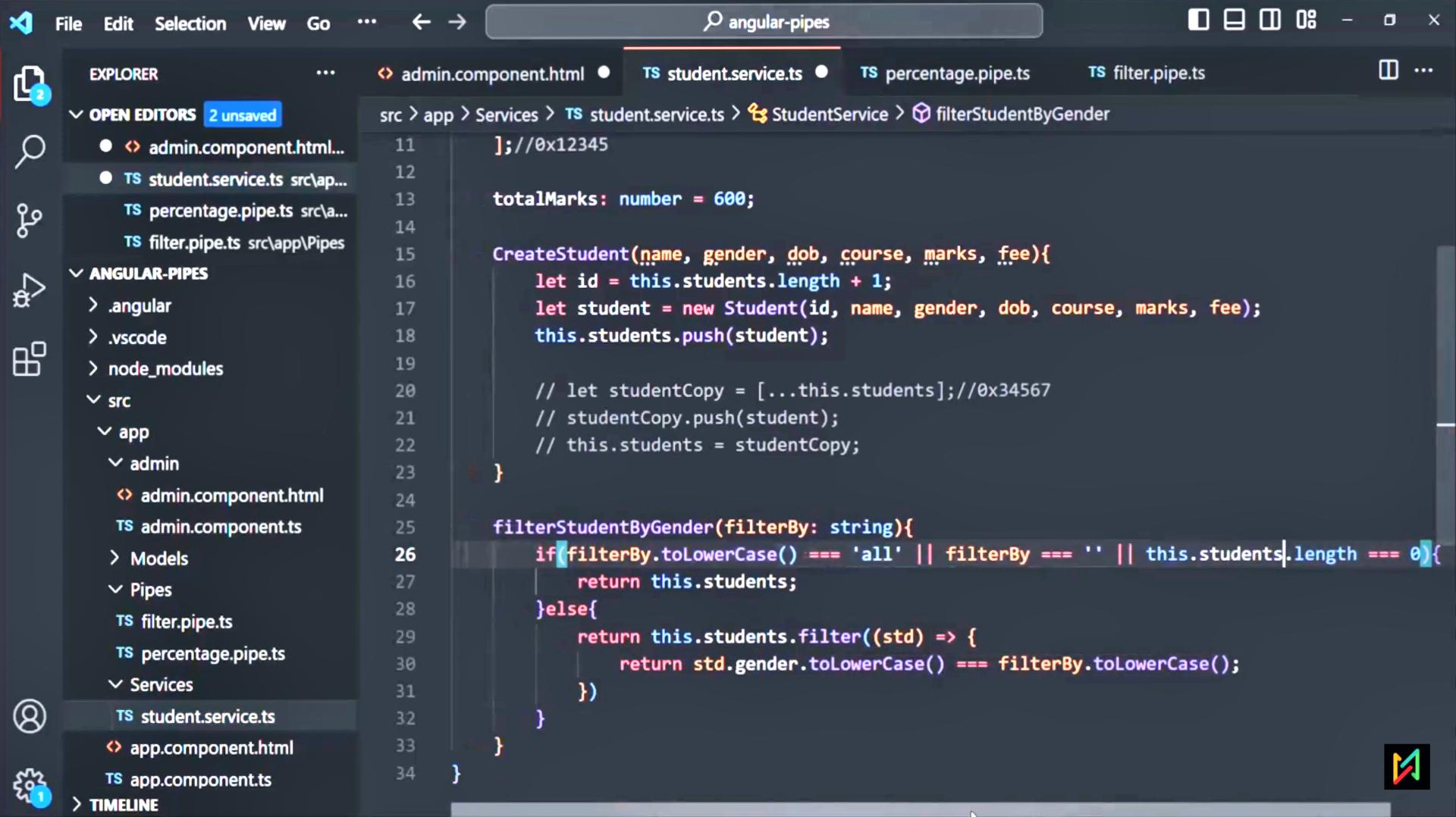
admin.component.html TS student.service.ts TS percentage.pipe.ts TS filter.pipe.ts

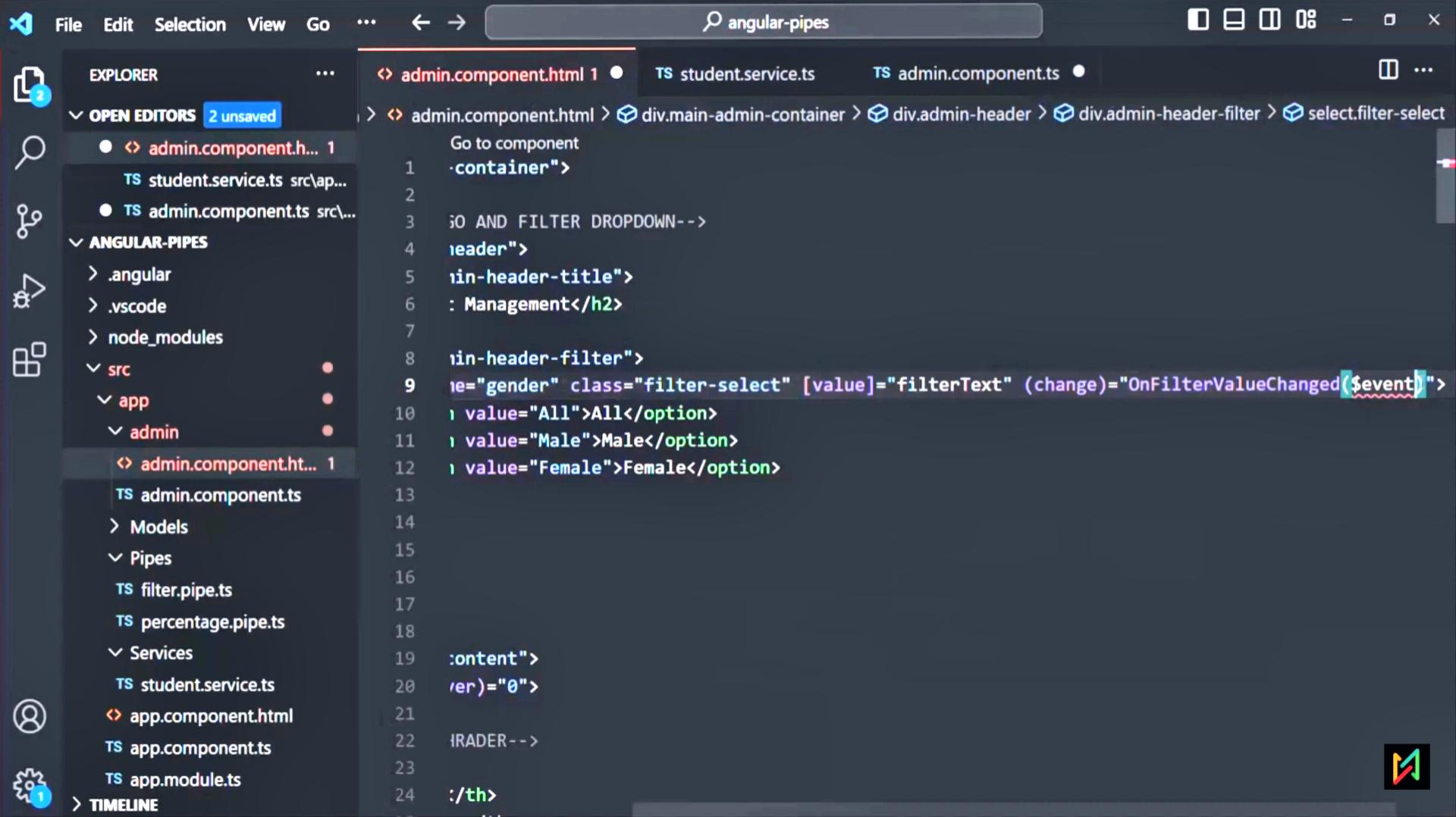
src > app > Pipes > TS filter.pipe.ts > FilterPipe

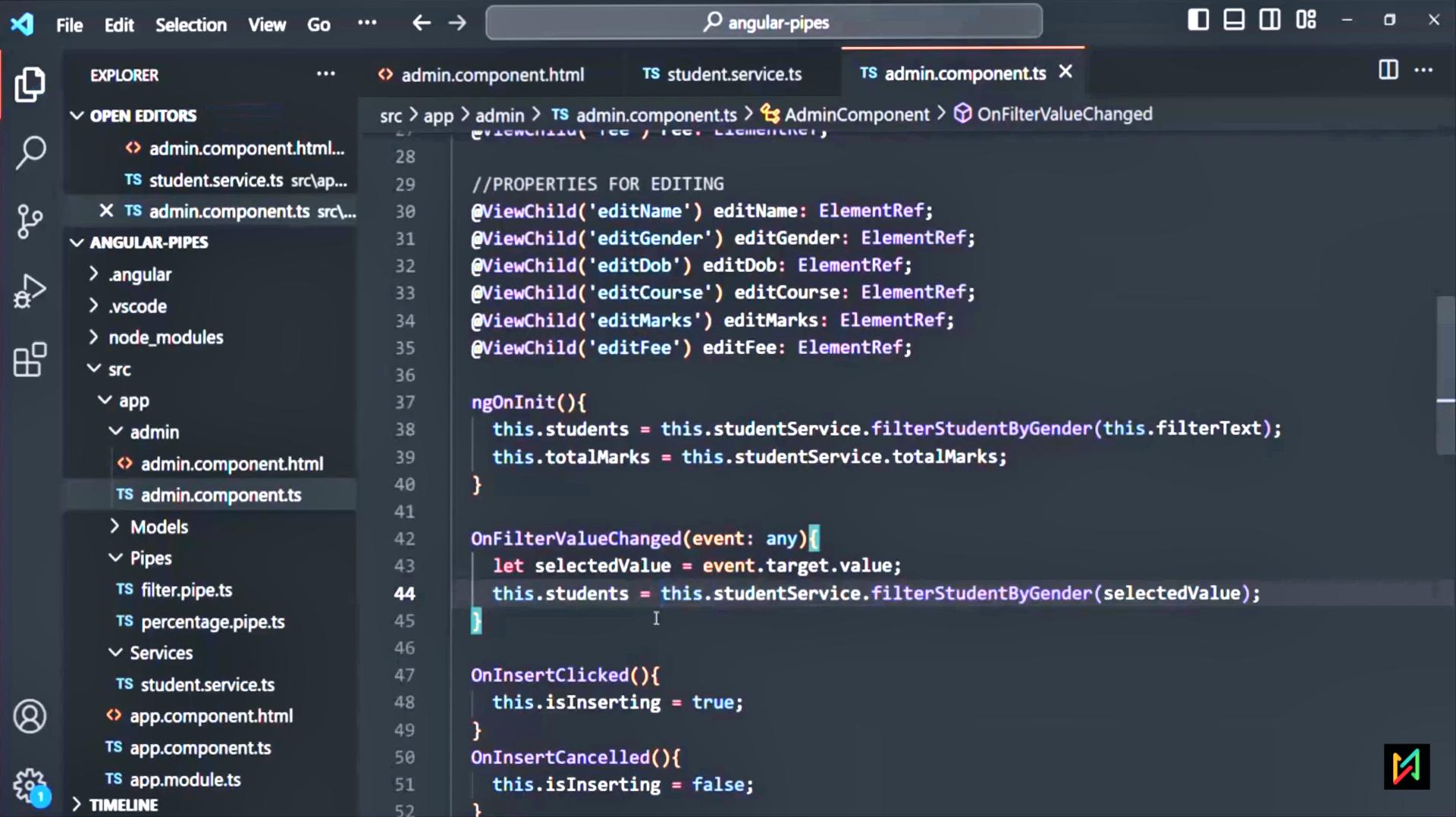
```
1 import { Pipe, PipeTransform } from "@angular/core";
2 import { Student } from "../Models/Student";
3
4 @Pipe({
5   name: 'filter',
6   pure: false
7 })
8 export class FilterPipe implements PipeTransform{
9   transform(list: Student[], filterBy: string) {
10     console.log('FILTER PIPE CALLED!');
11     if(filterBy.toLowerCase() === 'all' || filterBy === '' || list.length === 0){
12       return list;
13     }else{
14       return list.filter((std) => {
15         return std.gender.toLowerCase() === filterBy.toLowerCase();
16       })
17     }
18   }
19 }
```

Adding Filter Functionality without Pipes









angular-pipes

angular-pipes

File Edit Selection View Go ...

admin.component.html TS student.service.ts TS admin.component.ts

EXPLORED

OPEN EDITORS 1 unsaved

admin.component.html... TS student.service.ts src\ap... TS admin.component.ts src\...

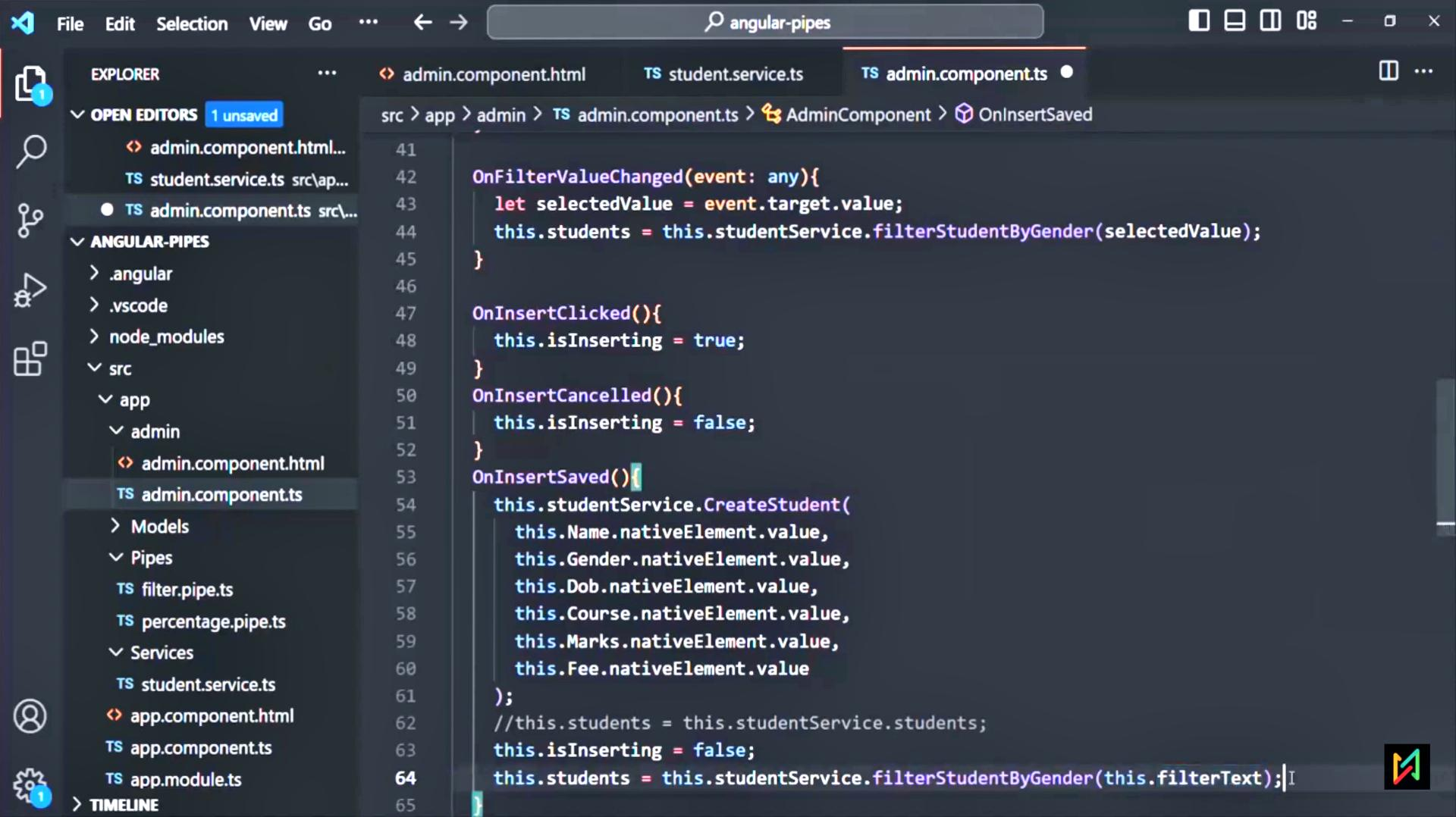
ANGULAR-PIPES

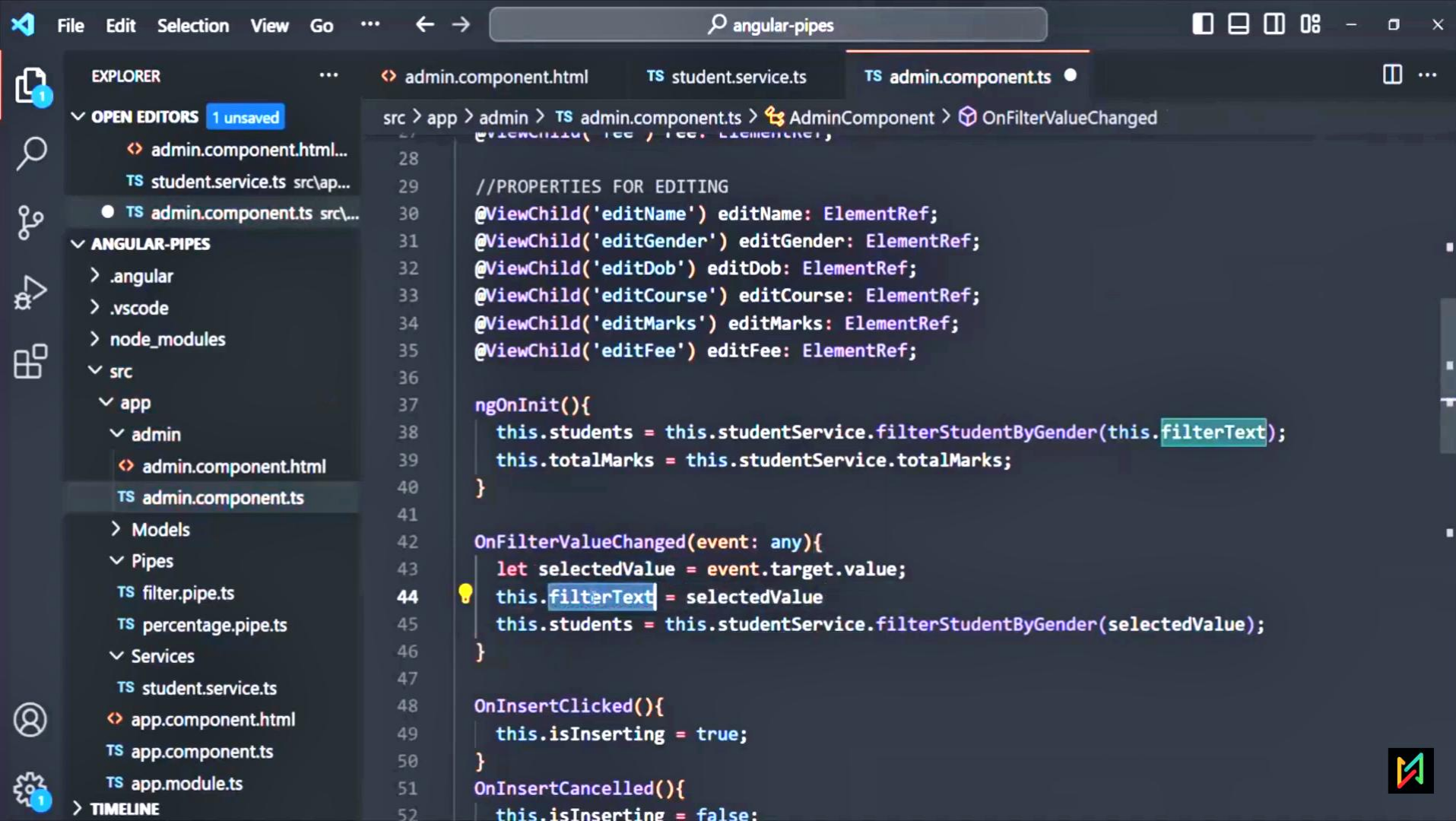
.angular .vscode node_modules src app admin admin.component.html TS admin.component.ts Models Pipes filter.pipe.ts percentage.pipe.ts Services student.service.ts app.component.html TS app.component.ts app.module.ts

TIMELINE

src > app > admin > TS admin.component.ts > AdminComponent > OnInsertSaved

```
41
42 OnFilterValueChanged(event: any){
43     let selectedValue = event.target.value;
44     this.students = this.studentService.filterStudentByGender(selectedValue);
45 }
46
47 OnInsertClicked(){
48     this.isInserting = true;
49 }
50 OnInsertCancelled(){
51     this.isInserting = false;
52 }
53 OnInsertSaved(){
54     this.studentService.CreateStudent(
55         this.Name.nativeElement.value,
56         this.Gender.nativeElement.value,
57         this.Dob.nativeElement.value,
58         this.Course.nativeElement.value,
59         this.Marks.nativeElement.value,
60         this.Fee.nativeElement.value
61     );
62     //this.students = this.studentService.students;
63     this.isInserting = false;
64     this.students = this.studentService.filterStudentByGender(this.filterText);
65 }
```

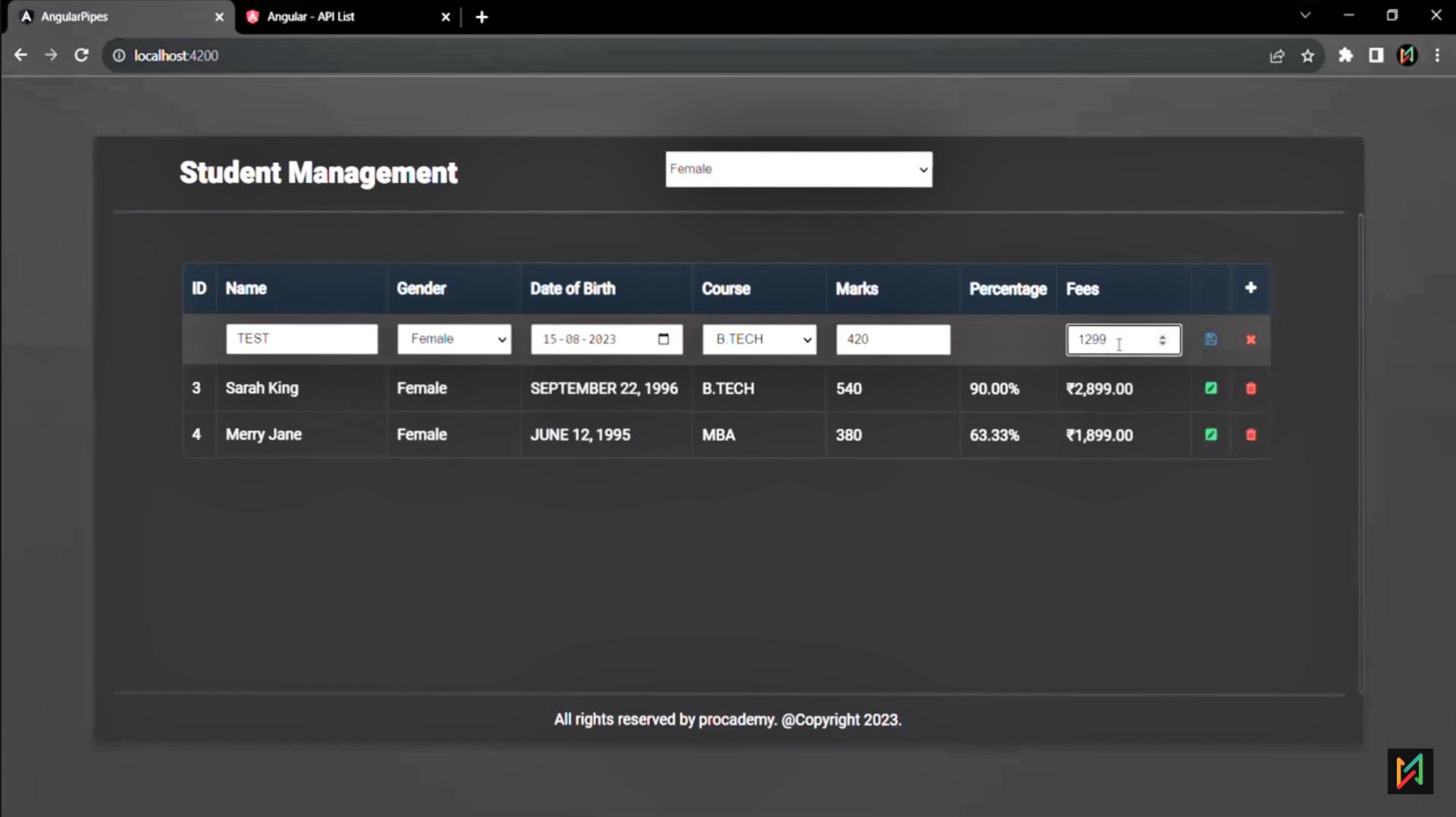


Student Management

Female

ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees		+
3	Sarah King	Female	SEPTEMBER 22, 1996	B.TECH	540	90.00%	₹2,899.00		
4	Merry Jane	Female	JUNE 12, 1995	MBA	380	63.33%	₹1,899.00		



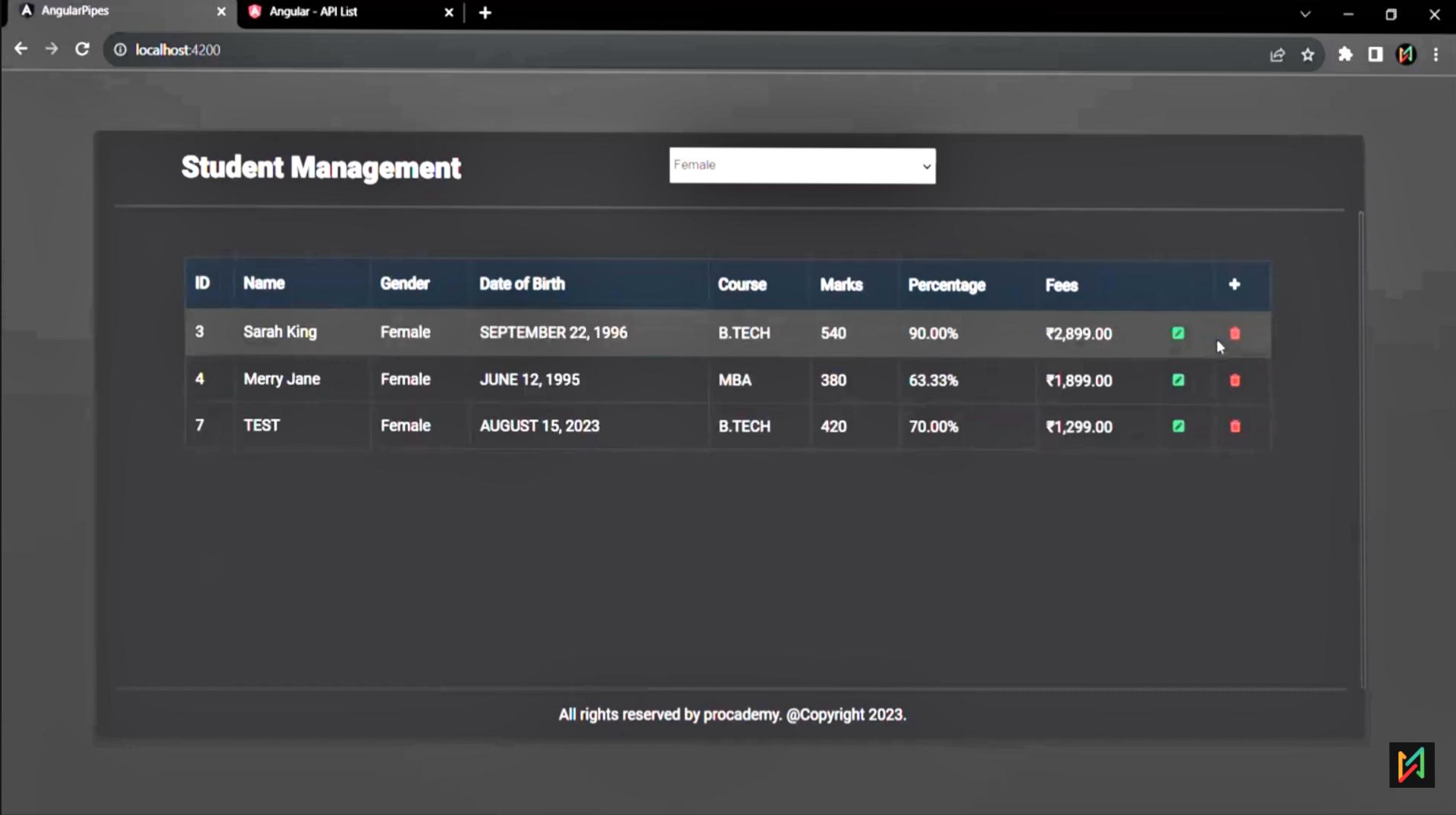


Student Management

Female

ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees		+
	TEST	Female	15 - 08 - 2023	B.TECH	420		1299		
3	Sarah King	Female	SEPTEMBER 22, 1996	B.TECH	540	90.00%	₹2,899.00		
4	Merry Jane	Female	JUNE 12, 1995	MBA	380	63.33%	₹1,899.00		

All rights reserved by procademy. @Copyright 2023.



Student Management

Female

ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees	+
3	Sarah King	Female	SEPTEMBER 22, 1996	B.TECH	540	90.00%	₹2,899.00	 
4	Merry Jane	Female	JUNE 12, 1995	MBA	380	63.33%	₹1,899.00	 
7	TEST	Female	AUGUST 15, 2023	B.TECH	420	70.00%	₹1,299.00	 

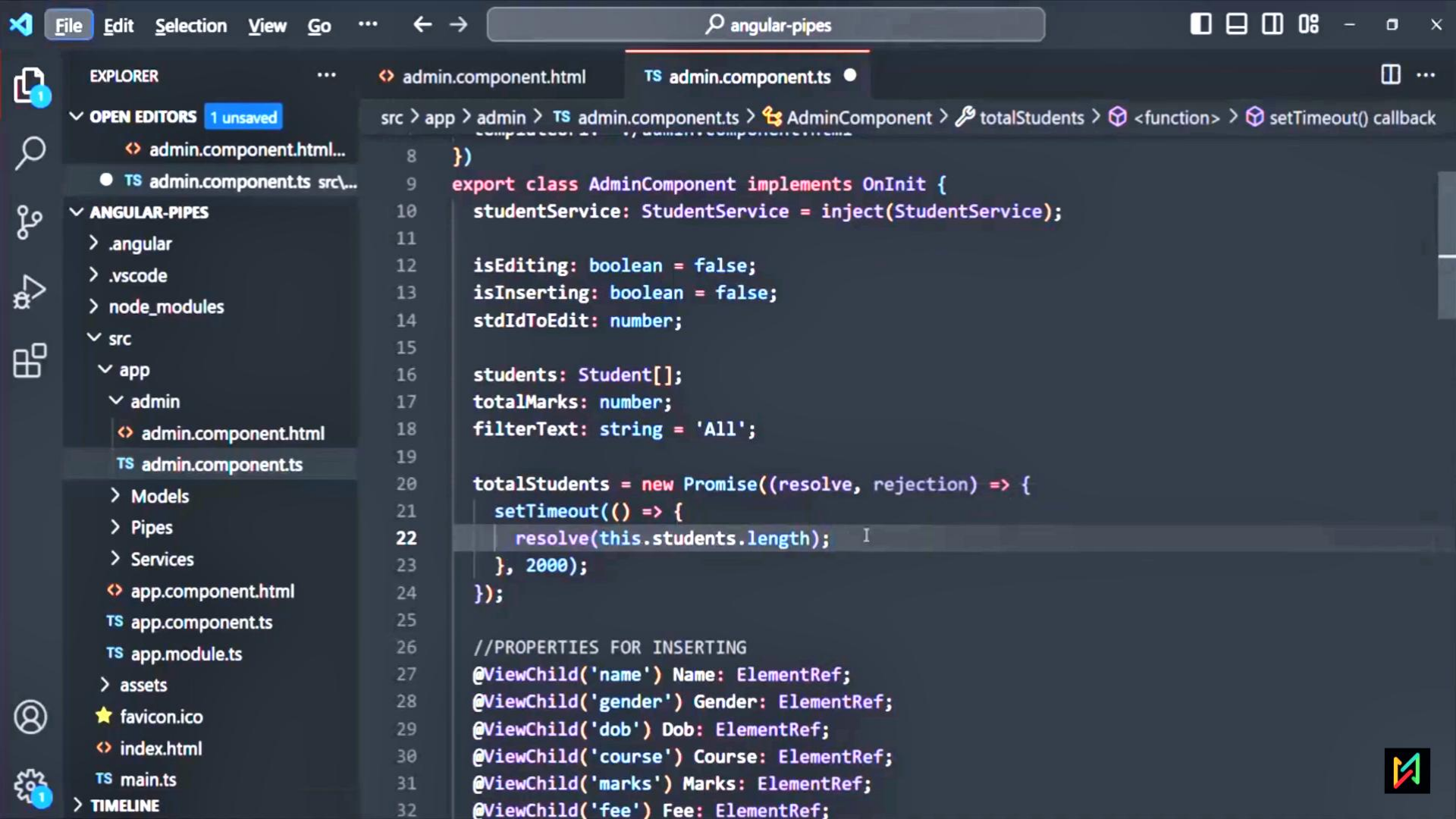
All rights reserved by procademy. @Copyright 2023.

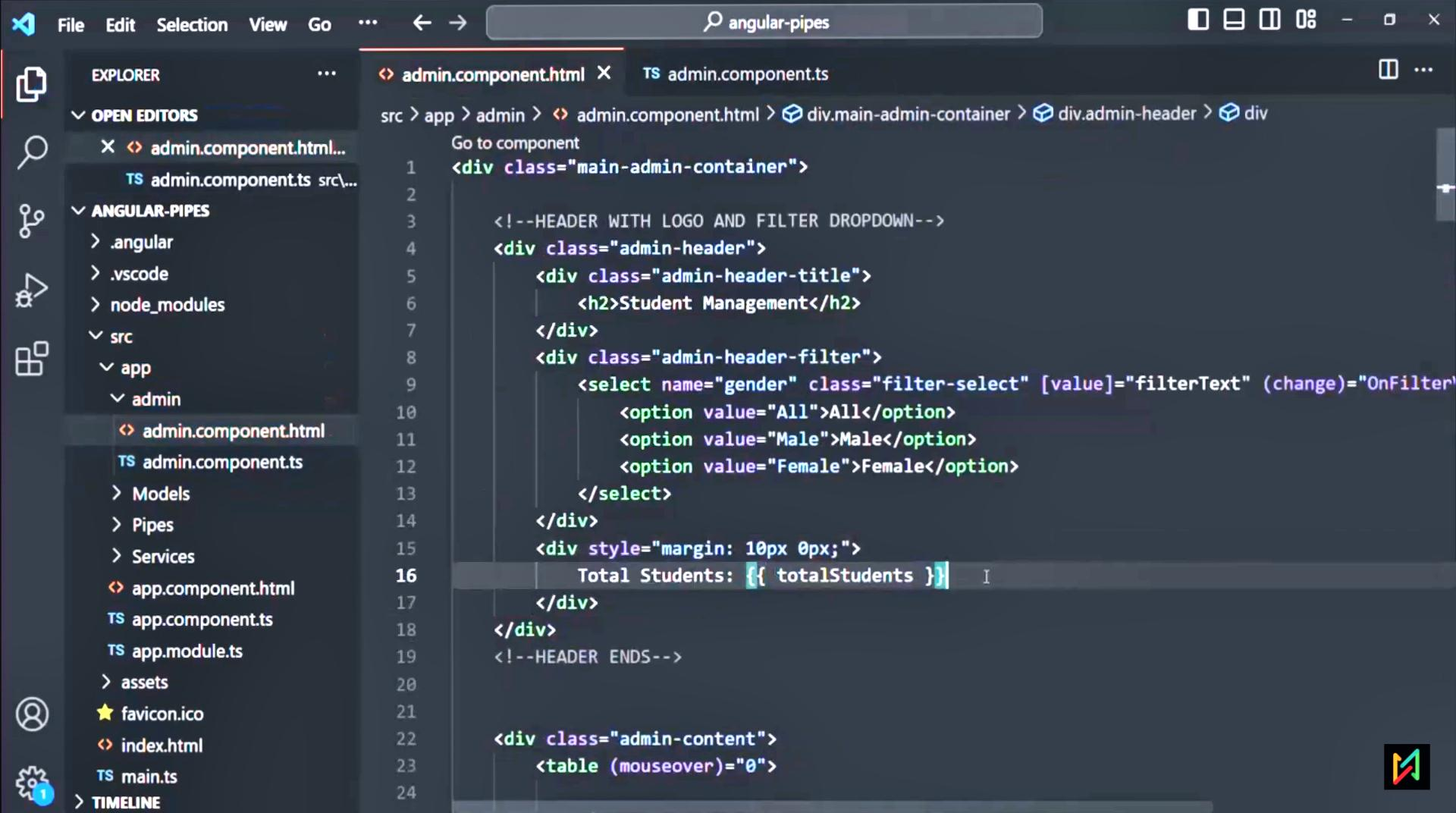


async pipe

The async pipe allows us to handle asynchronous data. It allows us to subscribe to an observable or promise from the view template and returns the value emitted.





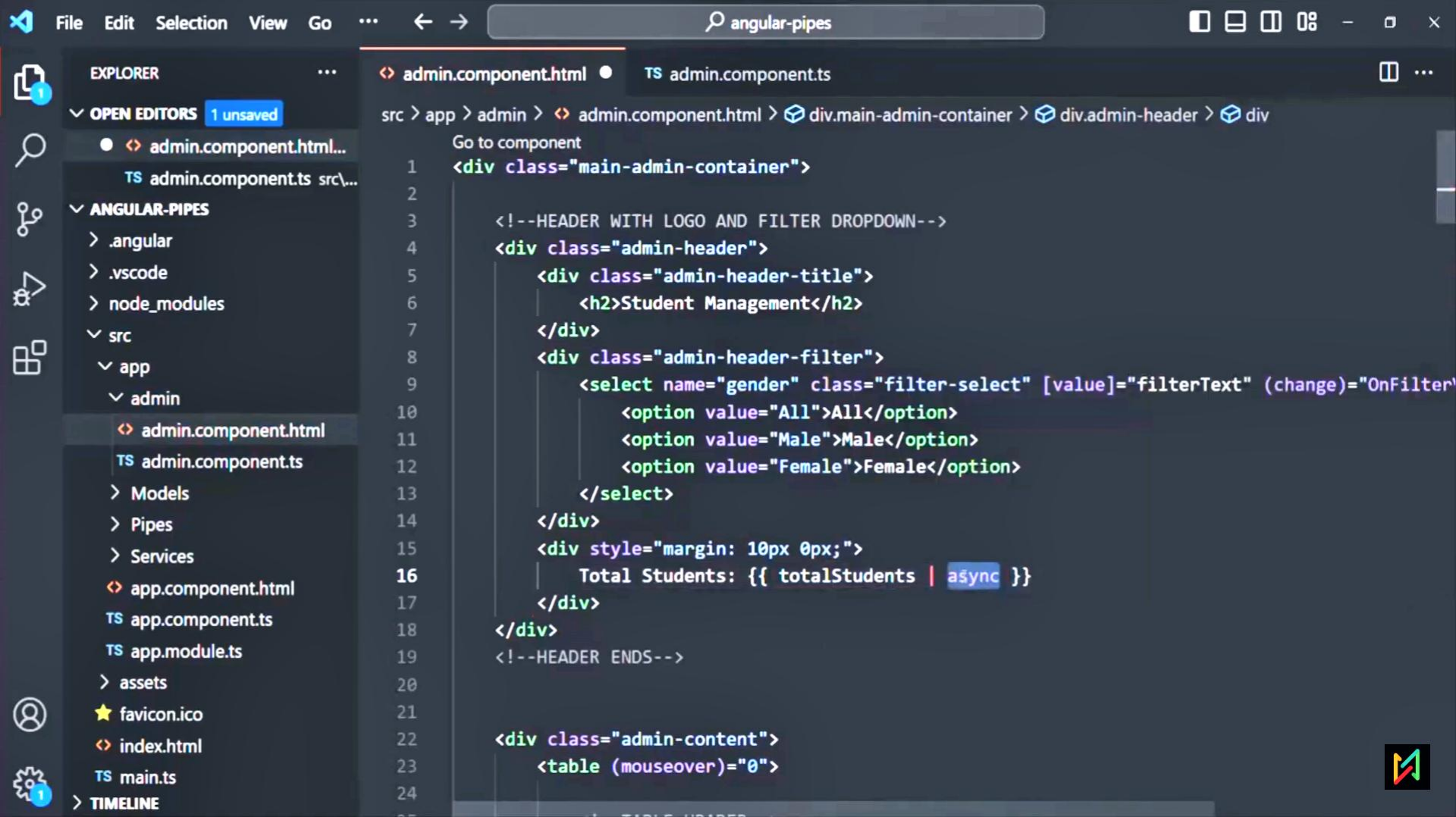


Student Management

All

Total Students: `object`
`Promise`

ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees		+
1	John Smith	Male	NOVEMBER 12, 1997	MBA	520	86.67%	₹1,899.00	☒	☐
2	Mark Vought	Male	OCTOBER 6, 1998	B.TECH	420	70.00%	₹2,899.00	☒	☐
3	Sarah King	Female	SEPTEMBER 22, 1996	B.TECH	540	90.00%	₹2,899.00	☒	☐
4	Merry Jane	Female	JUNE 12, 1995	MBA	380	63.33%	₹1,899.00	☒	☐
5	Steve Smith	Male	DECEMBER 21, 1999	M.SC	430	71.67%	₹799.00	☒	☐
6	Jonas Weber	Male	JUNE 18, 1997	M.SC	320	53.33%	₹799.00	☒	☐



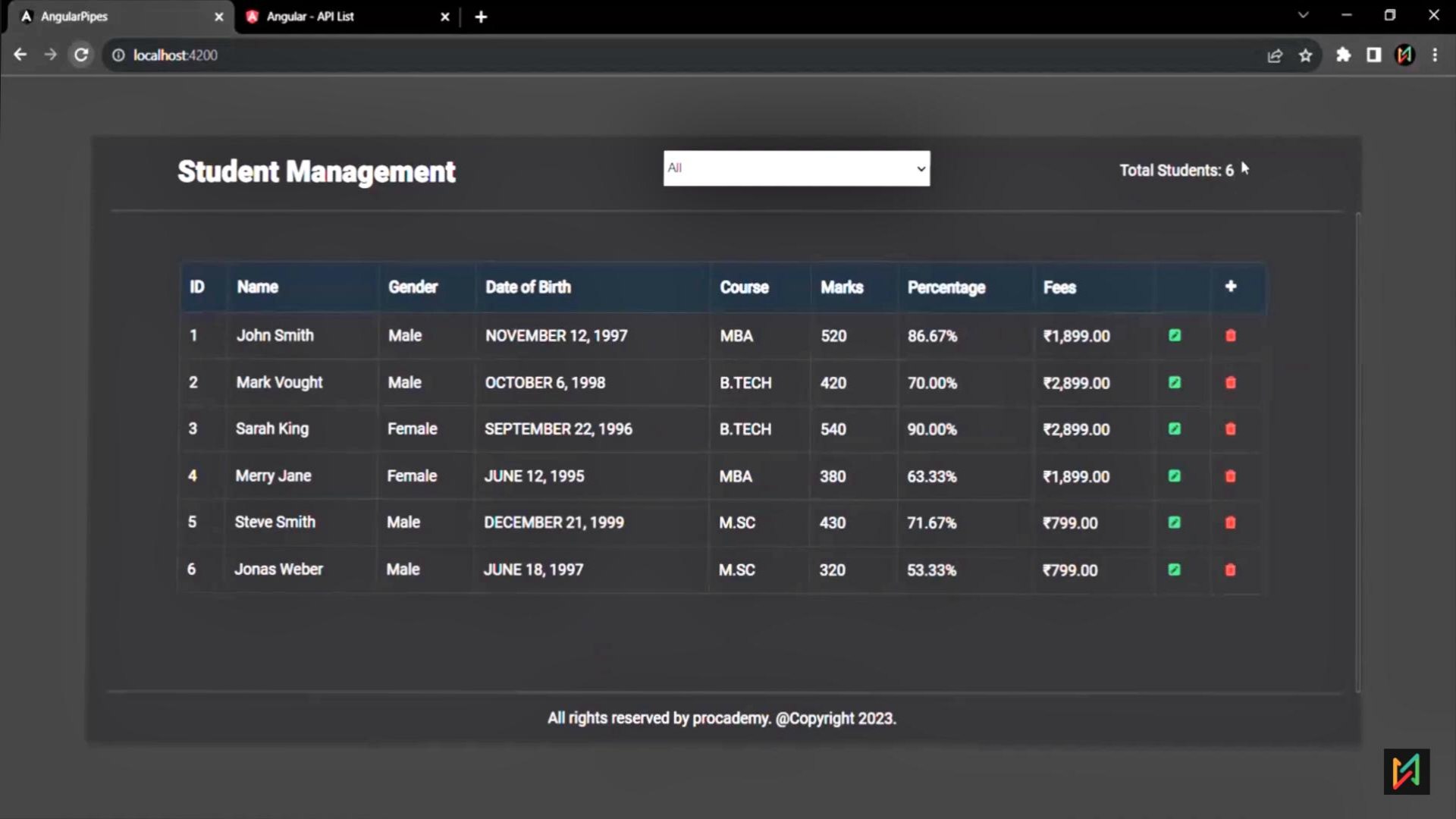
Student Management

All

Total Students:

ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees		+
1	John Smith	Male	NOVEMBER 12, 1997	MBA	520	86.67%	₹1,899.00	✓	✗
2	Mark Vought	Male	OCTOBER 6, 1998	B.TECH	420	70.00%	₹2,899.00	✓	✗
3	Sarah King	Female	SEPTEMBER 22, 1996	B.TECH	540	90.00%	₹2,899.00	✓	✗
4	Merry Jane	Female	JUNE 12, 1995	MBA	380	63.33%	₹1,899.00	✓	✗
5	Steve Smith	Male	DECEMBER 21, 1999	M.SC	430	71.67%	₹799.00	✓	✗
6	Jonas Weber	Male	JUNE 18, 1997	M.SC	320	53.33%	₹799.00	✓	✗





Student Management

All

Total Students: 6

ID	Name	Gender	Date of Birth	Course	Marks	Percentage	Fees		+
1	John Smith	Male	NOVEMBER 12, 1997	MBA	520	86.67%	₹1,899.00	☒	☐
2	Mark Vought	Male	OCTOBER 6, 1998	B.TECH	420	70.00%	₹2,899.00	☒	☐
3	Sarah King	Female	SEPTEMBER 22, 1996	B.TECH	540	90.00%	₹2,899.00	☒	☐
4	Merry Jane	Female	JUNE 12, 1995	MBA	380	63.33%	₹1,899.00	☒	☐
5	Steve Smith	Male	DECEMBER 21, 1999	M.SC	430	71.67%	₹799.00	☒	☐
6	Jonas Weber	Male	JUNE 18, 1997	M.SC	320	53.33%	₹799.00	☒	☐

All rights reserved by procademy. @Copyright 2023.

